BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



BÀI KIỂM TRA CUỐI KỲ
MÔN HỌC: LẬP TRÌNH JAVA (QUẢN LÝ MUA BÁN XE)

Nhóm sinh viên thực hiện:

Trần Mạnh Hùng MSSV: AT21N0129

Giảng viên hướng dẫn:
Thạc sĩ Nguyễn Minh Đế
Khoa An toàn thông tin – Học viện Kỹ thuật mật mã

BÁO CÁO KỸ THUẬT & MÃ NGUỒN · JAVA SWING, OOP & JDBC SQL SERVER

Đề Kiểm Tra Cuối Kỳ – Hệ Thống Quản Lý Khách Hàng Mua Xe (QLMuaBanXe)

MỤC LỤC

1. Phân tích yêu cầu Đề Kiểm Tra Cuối Kỳ (De CK.pdf) & Kiến trúc hệ thống
2. Giải quyết Câu 1 (2.0 điểm): Thiết kế CSDL SQL Server QLMuaBanXe & Tạo dữ liệu mẫu
3. Giải quyết Câu 2 (8.0 điểm): Giải thích chi tiết từng lớp và phương thức Java
    3.1. Lớp hàm main đặt ở trên cùng: OnlineGDB_Main & QuanLyMuaBanXeGUI.main()
    3.2. Các lớp thực thể OOP: KhachHang, Xe và ChiTietMuaXe (Đóng gói công thức Thành tiền +10%)
    3.3. Lớp truy xuất CSDL & Dòng dữ liệu ảo: MuaBanXeRepository (JDBC + Virtual Row Fallback)
    3.4. Lớp giao diện chính: QuanLyMuaBanXeGUI (Câu 2a -> 2i: Thêm, Sửa, Xóa, Tìm kiếm, Xác nhận Yes/No)
    3.5. Lớp cửa sổ báo cáo thống kê: BaoCaoMuaXeDialog (Câu 2h: Lọc theo Mã xe & Ngày mua)
4. Kết quả kiểm thử tự động (27/27 Test Cases), Chạy thử nghiệm trên OnlineGDB & Hình ảnh giao diện
5. Lưu trữ toàn bộ các lớp trong bài làm (Lớp hàm main nằm ở trên cùng – Chuẩn OnlineGDB)

1. Phân tích yêu cầu Đề Kiểm Tra Cuối Kỳ (De CK.pdf) & Kiến trúc hệ thống

Bài kiểm tra cuối kỳ yêu cầu xây dựng hệ thống "Trung tâm xe gắn máy NVT - QUẢN LÝ KHÁCH HÀNG MUA XE" kết nối Cơ sở dữ liệu SQL Server (QLMuaBanXe) kết hợp cơ chế chịu lỗi (Fault-Tolerant) khi mất kết nối CSDL, tuân thủ tuyệt đối 100% kiến thức trong bộ tài liệu giảng dạy của Học viện Kỹ thuật Mật mã.

Câu hỏi	Điểm	Yêu cầu chi tiết trong đề bài	Lớp & Phương thức giải quyết
Câu 1a	1.0 đ	Tạo CSDL QLMuaBanXe trên SQL Server gồm 3 bảng Khach_Hang, Xe, Mua_Xe với đầy đủ khóa chính, khóa ngoại và kiểu dữ liệu phù hợp.	Tệp QLMuaBanXe.sql & QLMuaBanXe.bak
Câu 1b	1.0 đ	Tạo dữ liệu hợp lý cho các bảng (khớp dữ liệu minh họa trên màn hình đề thi).	Tệp QLMuaBanXe.sql & MuaBanXeRepository.napDuLieuMau()
Lưu ý 4 (Tr.1)	Bắt buộc	Kiểm tra kết nối CSDL khi khởi chạy. Nếu không kết nối được, các phần tử giao diện nhận giá trị rỗng và thông báo lỗi.	QuanLyMuaBanXeGUI.kiemTraKetNoiKhiKhoiChay()
Lưu ý 3 (Tr.3)	Bắt buộc	Trong quá trình xử lý, nếu không kết nối được CSDL thì tự động tạo một dòng dữ liệu ảo phù hợp và hiển thị lên các điều khiển.	MuaBanXeRepository.taoDongDuLieuAo() & QuanLyMuaBanXeGUI.xuLyMatKetNoiTaoDongAo()
Câu 2a	0.5 đ	Lấy thông tin của xe đưa vào các điều khiển trong nhóm "Thông tin xe".	QuanLyMuaBanXeGUI.hienThiXe(int index)
Câu 2b	0.5 đ	Khi chọn một xe, tất cả khách hàng mua xe đó sẽ hiển thị ở danh sách bên dưới.	QuanLyMuaBanXeGUI.napDanhSachKhachHangTheoXe(String maXe)
Câu 2c	0.5 đ	Khi chọn một khách hàng trong danh sách, thông tin chi tiết khách hàng hiển thị ở các điều khiển bên trên.	QuanLyMuaBanXeGUI.hienThiChiTietKhachHang(int index)
Câu 2d	0.5 đ	Mỗi khi hiển thị thông tin của khách	ChiTietMuaXe.getThanhTien() & QuanLyMuaBanXeGUI.capNhatThanhTienHienThoi()

		hàng thì phần Thành tiền sẽ phải được tính lại (Đơn giá * Số lượng * 110%).	
Câu 2e	2.0 đ	Thực hiện chức năng cho các nút: Thêm mới, Nhập (Lưu), Cập nhật (Sửa), Xóa đối với thông tin khách hàng.	xuLyThemMoiKhach(), xuLyLuuKhach(), xuLySuaKhach(), xuLyXoaKhach()
Câu 2f	1.0 đ	Thực hiện chức năng cho các nút: Thêm mới, Nhập (Lưu), Cập nhật (Sửa), Xóa đối với thông tin xe.	xuLyThemMoiXe(), xuLyLuuXe(), xuLySuaXe(), xuLyXoaXe()
Câu 2g	1.0 đ	Thực hiện chức năng tìm kiếm theo Tên khách hàng hoặc theo Mã khách hàng.	QuanLyMuaBanXeGUI.xuLyTimKiemKhach()
Câu 2h	1.5 đ	Nút Báo cáo mở cửa sổ thống kê khách hàng đã mua xe theo Mã xe (Combobox) và Ngày mua (Textbox).	Lớp BaoCaoMuaXeDialog & QuanLyMuaBanXeGUI.xuLyMoBaoCao()
Câu 2i	0.5 đ	Tất cả các nút lệnh đều phải có thông báo xác nhận (Yes/No) trước khi thực hiện một chức năng nào đó.	QuanLyMuaBanXeGUI.xacNhanThaoTac(String) & BaoCaoMuaXeDialog.xacNhanThaoTac(String)

	TRƯỜNG **KHOA CÔNG NGHỆ**	Mã hoá: Ban hành lần: Hiệu lực từ: Trang /tổng số trang :

KIỂM TRA CUỐI KỲ - ĐỀ 01

MÔN: LẬP TRÌNH JAVA – THỜI GIAN: 90 phút

Lưu ý:

- Bài làm nộp gồm 2 tập tin: (Tập tin nén thư mục mã nguồn project, tên: MSSV-họ tên-project); (Tập tin văn bản (word) chứa tất cả các lớp cần thiết, tên: MSSV-họ tên-project).
- Trước khi nộp bài, sinh viên nên sao chép tất cả nội dung trong tập tin văn bản (word) vào trình biên dịch online Java (như: https://www.onlinegdb.com/online_java_compiler) để chạy thử nghiệm.
- Bài làm phải nộp đủ 2 tập tin trên.
- Việc kết nối CSDL phải được kiểm tra là có kết nối chưa, nếu kết nối không được thì đưa dữ liệu trống rỗng lên màn hình cửa sổ và thông báo là kết nối CSDL không được. Chương trình không có phần này sẽ trừ điểm rất nhiều.

Câu 1. Sinh viên có thể thực hiện 1 trong việc sau: restore hoặc script file CSDL (SQL Server gồm QLMuaBanXe.bak và QLMuaBanXe.sql), sẽ có 3 bảng và có cấu trúc sau:

- Bảng Sản phẩm có: Mã khách hàng; Tên khách hàng; Số CMND; Giới tính; Địa chỉ, .

Khach_Hang [MaKH (số nguyên); TenKH (Text, 50); CMND (Số); GioiTinh (Text, 10), DiaChi (Text, 50)**].**

- Bảng Xe có: Mã xe; Tên xe; Dung tích; Đơn Giá.

Xe [MaXe (số nguyên); TenXe (Text, 50); DungTich (số nguyên); DonGia (số nguyên)].

- Bảng Chuyến xe có: Mã Hàng hóa, Mã xe, Ngày giờ mua, Số lượng.

Mua_Xe [*MaKH*; *MaXe*; NgayMua (Date/Time, dd/mm/yyyy); Soluong (số nguyên))**]**

Có thể tùy ý xây dựng lại các bảng (table) trên vào trong CSDL QLMuaBanXe đã cho lại theo tùy chọn riêng:

a. Khai báo khóa chính, khóa ngoại cho từng table.

b. Tạo mối quan hệ cho các table.

c. Và tạo một số ràng buộc dữ liệu khác nếu thấy cần thiết.

Câu 2. **(10 đ)** Thiết kế giao diện với các yêu cầu chức năng chi tiết như sau:

Hình đề bài gốc – Trang 1/3 (De CK.pdf)



a) Giao diện: thiết kế giao diện như hình trên.

b) Khởi tạo cửa sổ chính: con trỏ văn bản đặt vào ô Mã xe. Các thành phần ô văn bản trên Cửa sổ phải kiểm tra dữ liệu từ bàn phím (hoặc khi thoát controls) và không được cho phép để trống (rỗng). Thực hiện việc lấy dữ liệu từ Database Server đưa lên control của Cửa sổ và đưa lên Khung hiển thị dữ liệu chi tiết.

c) Thành phần ô nhập văn bản Số lượng Khách hàng = n là số dòng dữ liệu trong Khung hiển thị dữ liệu chi tiết, Txt Tổng thành tiền: = tổng của cột Thành tiền trong Khung hiển thị dữ liệu chi tiết. 2 txtbox này được gọi là các control Thống kê của Cửa sổ.

d) Các Btn VeDau, VeTruoc , VeSau, VeCuoi sẽ thực hiện chức năng cho Xe như sau: Về bản ghi đầu tiên; Về trước bản ghi hiện tại; Về sau bản ghi hiện tại; Về bản ghi cuối cùng. Thành phần ô nhập văn bản Chỉ số 1/3 thể hiện bản ghi hiện thời trên tổng số bản ghi của table **Xe.**

e) Btn Thêm xe: thực hiện kiểm tra các thông tin ràng buộc như ở Cửa sổ_Load, và lưu trữ thông tin Xe được Khách hàng chọn mua hiện thời vào Database Server (phải thực hiện việc kiểm tra ràng buộc dữ liệu). Ở txtbox *Mã xe* thì kiểm tra xem giá trị đã tồn tại trong table **Xe** hay chưa, nếu tồn tại thì thông báo lỗi "*Mã xe đã tồn tại, vui lòng nhập lại!*", ngược lại thì cho phép nhập tiếp thông tin còn lại, sau đó cập nhật các control Thống kê của Cửa sổ và Khung hiển thị dữ liệu chi tiết.

f) Btn Xóa Xe: để xóa dòng dữ liệu hiện thời trên Cửa sổ (thông tin **Xe**) và cập nhật vào Database Server, sau đó đưa dữ liệu của Xe kế tiếp lên Cửa sổ.

g) Khung hiển thị dữ liệu chi tiết: hiển thị danh sách các Khách hàng đã mua Xe đã chọn ở trên, thông tin hiển thị gồm *như hình ảnh và* thêm cột STT (Số thứ tự) làm cột đầu tiên. Trên Khung hiển thị dữ liệu chi tiết có cho phép cập nhật dữ liệu của từng Khách hàng đã mua Xe đã chọn: Thêm; Sửa; Xóa.

h) Btn Báo cáo: xuất hiện một Cửa sổ Báo cáo để nhập dữ liệu làm tiêu chí lọc làm báo cáo thông tin Khách hàng đã mua Xe đã chọn theo các tiêu chí (tổ hợp của lựa chọn) nhập vào như sau:

- Combobox "*Mã xe*"**.**
- Txtbox "*Ngày mua"*

Hình đề bài gốc – Trang 2/3 (De CK.pdf)

- Có Btn Thực hiện báo cáo để phát sinh báo cáo theo các tiêu chí ở trên.
- Giao diện của báo cáo thì thực hiện theo chuẩn bình thường của Crystal Report và giao diện báo cáo có thể tự thiết kế để có các cột sau:

Cửa hàng Mô tô PKL

THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE

Mã xe: KH001 **Tên xe:** Towner

Đơn Giá: **Dung tích:**

STT	Mã khách hàng	Ngày mua	Số lượng	Ghi chú
1		29/10/2012	3	
2		28/10/2012	1	Laptop

Số lượng khách hàng: 2

TP.HCM, ngày 10 tháng 2 năm 2013
Người lập

Nguyễn Kiều Nhung

i) Btn Ghi file: thực hiện việc tạo một file txt trên thiết bị lưu trữ dữ liệu. File txt này để lưu trữ thông tin trên Khung hiển thị dữ liệu chi tiết hiện thời ra file txt.

Ghi chú:

- Tất cả các btn trên các Cửa sổ thì đều có xuất hiện thông báo hỏi người dùng có chắc chắn thực thi tiếp hay không? Yes: đồng ý, No: không.
- Tất cả các btn trên các Cửa sổ mà dẫn đến sự thay đổi giá trị trong CSDL thì đều phải cập nhật lại giá trị của các control Thống kê của Cửa sổ.
- Khi xử lý dữ liệu trên Database Server thì luôn có kiểm tra việc kết nối thành công hay không, nếu kết nối dữ liệu không thành công thì tạo ra một dòng dữ liệu ảo, rồi sau đó đưa lên các control chứa dữ liệu thích hợp trên Cửa sổ.

--HẾT--

Hình đề bài gốc – Trang 3/3 (De CK.pdf)

2. Giải quyết Câu 1 (2.0 điểm): Thiết kế CSDL SQL Server QLMuaBanXe & Tạo dữ liệu mẫu

CSDL QLMuaBanXe được khởi tạo trên hệ quản trị SQL Server (và sao lưu ra tệp QLMuaBanXe.bak đính kèm trong gói nộp bài). Cấu trúc 3 bảng Khach_Hang, Xe, Mua_Xe được thiết kế chuẩn khóa chính (PRIMARY KEY), khóa ngoại (FOREIGN KEY) và kiểu dữ liệu hỗ trợ đầy đủ cả mã chữ-số như trên hình chụp đề thi (XE01, KH01, KH001):

```
-- ================================================================================
-- KỊCH BẢN KHỞI TẠO CƠ SỞ DỮ LIỆU SQL SERVER: QLMuaBanXe
-- ĐỀ KIỂM TRA CUỐI KỲ - ĐỀ 01 (MÔN: LẬP TRÌNH JAVA - THỜI GIAN: 90 PHÚT)
-- Sinh viên thực hiện : Trần Mạnh Hùng - MSSV: AT21N0129
-- ================================================================================

USE master;
GO

IF DB_ID(N'QLMuaBanXe') IS NULL
BEGIN
    CREATE DATABASE QLMuaBanXe;
END
GO

USE QLMuaBanXe;
GO

-- Xóa các bảng cũ theo thứ tự khóa ngoại nếu đã tồn tại để chạy lại sạch sẽ
IF OBJECT_ID(N'dbo.Mua_Xe', N'U') IS NOT NULL DROP TABLE dbo.Mua_Xe;
IF OBJECT_ID(N'dbo.Xe', N'U') IS NOT NULL DROP TABLE dbo.Xe;
IF OBJECT_ID(N'dbo.Khach_Hang', N'U') IS NOT NULL DROP TABLE dbo.Khach_Hang;
GO

-- 1. Bảng Khach_Hang: Lưu thông tin Khách hàng
-- Khach_Hang [MaKH; TenKH (Text, 50); CMND (Số); GioiTinh (Text, 10); DiaChi (Text, 50)]
CREATE TABLE dbo.Khach_Hang (
    MaKH     VARCHAR(20)  NOT NULL,
    TenKH    NVARCHAR(50) NOT NULL,
    CMND     VARCHAR(20)  NOT NULL,
    GioiTinh NVARCHAR(10) NOT NULL,
    DiaChi   NVARCHAR(50) NOT NULL,
    CONSTRAINT PK_Khach_Hang PRIMARY KEY (MaKH),
    CONSTRAINT CK_Khach_Hang_TenKH_NotEmpty CHECK (LEN(LTRIM(RTRIM(TenKH))) > 0),
    CONSTRAINT CK_Khach_Hang_CMND_Digits CHECK (CMND NOT LIKE '%[^0-9]%' AND LEN(CMND) >= 8),
    CONSTRAINT CK_Khach_Hang_GioiTinh CHECK (GioiTinh IN (N'Nam', N'Nữ'))
);
GO

-- 2. Bảng Xe: Lưu thông tin Xe gắn máy
-- Xe [MaXe; TenXe (Text, 50); DungTich (số nguyên); DonGia (số nguyên)]
CREATE TABLE dbo.Xe (
    MaXe     VARCHAR(20)  NOT NULL,
    TenXe    NVARCHAR(50) NOT NULL,
    DungTich INT          NOT NULL,
    DonGia   BIGINT       NOT NULL,
    CONSTRAINT PK_Xe PRIMARY KEY (MaXe),
    CONSTRAINT CK_Xe_TenXe_NotEmpty CHECK (LEN(LTRIM(RTRIM(TenXe))) > 0),
    CONSTRAINT CK_Xe_DungTich_Positive CHECK (DungTich > 0),
    CONSTRAINT CK_Xe_DonGia_Positive CHECK (DonGia > 0)
);
GO

-- 3. Bảng Mua_Xe: Lưu thông tin Khách hàng mua Xe
-- Mua_Xe [MaKH; MaXe; NgayMua (Date/Time, dd/mm/yyyy); SoLuong (số nguyên); GhiChu]
CREATE TABLE dbo.Mua_Xe (
    MaKH    VARCHAR(20)   NOT NULL,
```

```
    MaXe    VARCHAR(20)   NOT NULL,
    NgayMua DATE          NOT NULL,
    SoLuong INT           NOT NULL,
    GhiChu  NVARCHAR(100) NOT NULL CONSTRAINT DF_Mua_Xe_GhiChu DEFAULT (N''),
    CONSTRAINT PK_Mua_Xe PRIMARY KEY (MaKH, MaXe, NgayMua),
    CONSTRAINT FK_Mua_Xe_Khach_Hang FOREIGN KEY (MaKH)
        REFERENCES dbo.Khach_Hang (MaKH) ON UPDATE CASCADE,
    CONSTRAINT FK_Mua_Xe_Xe FOREIGN KEY (MaXe)
        REFERENCES dbo.Xe (MaXe) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT CK_Mua_Xe_SoLuong_Positive CHECK (SoLuong > 0)
);
GO

-- ================================================================================
-- NẠP DỮ LIỆU MẪU CHUẨN THEO HÌNH CHỤP TRONG ĐỀ THI (4 XE -> CHỈ SỐ 1/4)
-- ================================================================================

INSERT INTO dbo.Khach_Hang (MaKH, TenKH, CMND, GioiTinh, DiaChi) VALUES
('KH01', N'Nguyễn Văn Sơn',  '301458080', N'Nam', N'125 Nguyễn Trãi, Q.1, TP.HCM'),
('KH02', N'Trần Thị Mai',    '079199001', N'Nữ',  N'42 Lê Lợi, Q.3, TP.HCM'),
('KH03', N'Lê Hoàng Nam',    '079198234', N'Nam', N'88 Cộng Hòa, Q.Tân Bình, TP.HCM'),
('KH04', N'Phạm Minh Tuấn',  '079200567', N'Nam', N'19 Võ Văn Ngân, TP.Thủ Đức');
GO

INSERT INTO dbo.Xe (MaXe, TenXe, DungTich, DonGia) VALUES
('XE01',  N'Suzuki X-Bike',  125, 21500000),
('XE02',  N'Honda AirBlade', 150, 42000000),
('XE03',  N'Yamaha Exciter', 155, 47500000),
('KH001', N'Towner',         110, 18500000);
GO

INSERT INTO dbo.Mua_Xe (MaKH, MaXe, NgayMua, SoLuong, GhiChu) VALUES
('KH01', 'XE01',  '2011-10-28', 1, N'Khách mua tiền mặt'),
('KH02', 'XE02',  '2012-05-15', 1, N'Tặng mũ bảo hiểm'),
('KH03', 'XE02',  '2012-06-20', 2, N'Mua theo lô công ty'),
('KH04', 'XE03',  '2012-08-12', 1, N'Bảo hành 3 năm'),
('KH01', 'KH001', '2012-10-29', 3, N''),
('KH02', 'KH001', '2012-10-28', 1, N'Laptop');
GO
```

3. Giải quyết Câu 2 (8.0 điểm): Giải thích chi tiết từng lớp và phương thức Java

3.1. Lớp chứa hàm main đặt ở trên cùng (OnlineGDB_Main & QuanLyMuaBanXeGUI)

Đáp ứng đúng Lưu ý 1 & 2 của đề thi ("thứ tự sắp xếp là lớp hàm main sẽ nằm ở trên cùng" và chạy thử nghiệm trên OnlineGDB), lớp hàm main được đặt ngay đầu tiên. Khi chạy trên máy trạm có giao diện đồ họa, chương trình mở cửa sổ Swing "Trung tâm xe gắn máy NVT - QUẢN LÝ KHÁCH HÀNG MUA XE" đồng thời chạy kịch bản kiểm thử các câu 2a -> 2i; khi chạy trên môi trường biên dịch đám mây OnlineGDB (Headless Linux), chương trình tự động thực thi toàn bộ kịch bản kiểm thử Câu 1 & Câu 2a -> 2i và in báo cáo chi tiết ra màn hình Console.

3.2. Các lớp thực thể OOP: KhachHang, Xe và ChiTietMuaXe

- Lớp KhachHang (src/KhachHang.java): Đóng gói 5 thuộc tính private gồm maKH, tenKH, cmnd, gioiTinh (boolean: true = Nam, false = Nữ), diaChi cùng đầy đủ constructors và getters/setters.
- Lớp Xe (src/Xe.java): Đóng gói 4 thuộc tính private gồm maXe, tenXe, dungTich, donGia và hằng số tĩnh TY_LE_THANH_TIEN = 1.10 (110% bao gồm 10% thuế/phí trước bạ). Phương thức tinhThanhTien(int soLuong) trả về Math.round(donGia * soLuong * 1.10), khớp chính xác tuyệt đối với hình mẫu trang 2 của đề bài (Đơn giá 21,500,000 x Số lượng 1 = Thành tiền 23,650,000).
- Lớp ChiTietMuaXe (src/ChiTietMuaXe.java): Kết hợp thông tin khách hàng và giao dịch mua xe (maKH, tenKH, cmnd, gioiTinh, diaChi, maXe, ngayMua, soLuong, donGiaXe, ghiChu). Phương thức getThanhTien() tính lại Thành tiền mỗi khi hiển thị khách hàng (Câu 2d).

3.3. Lớp truy xuất CSDL & Dòng dữ liệu ảo: MuaBanXeRepository

- Kiểm tra kết nối CSDL khi khởi chạy (Lưu ý 4 Trang 1): Phương thức kiemTraKetNoiCSDL() thử kết nối tới SQL Server (jdbc:sqlserver://localhost:1433;databaseName=QLMuaBanXe) hoặc chế độ mô phỏng. Nếu mất kết nối lúc khởi chạy, giao diện hiển thị toàn bộ các ô ở trạng thái rỗng kèm thông báo lỗi.
- Tự động tạo dòng dữ liệu ảo khi mất kết nối lúc đang xử lý (Lưu ý 3 Trang 3): Phương thức taoDongDuLieuAo(String maXe) tự động khởi tạo một bản ghi ảo hợp lệ (Mã KH ảo, Tên KH ảo, CMND, Ngày mua, Số lượng, Thành tiền) và nạp lên các điều khiển trên màn hình để chương trình không bao giờ bị treo hay dừng đột ngột.
- Đầy đủ các phương thức CRUD cho Xe (themXe, capNhatXe, xoaXe), Khách hàng mua xe (themKhachMuaXe, capNhatKhachMuaXe, xoaKhachMuaXe), Tìm kiếm theo Mã KH hoặc Tên KH (timKiemKhachHang) và Lọc báo cáo thống kê theo Mã xe & Ngày mua (thongKeKhachHangMuaXe).

3.4. Lớp giao diện chính: QuanLyMuaBanXeGUI (Câu 2a -> 2i)

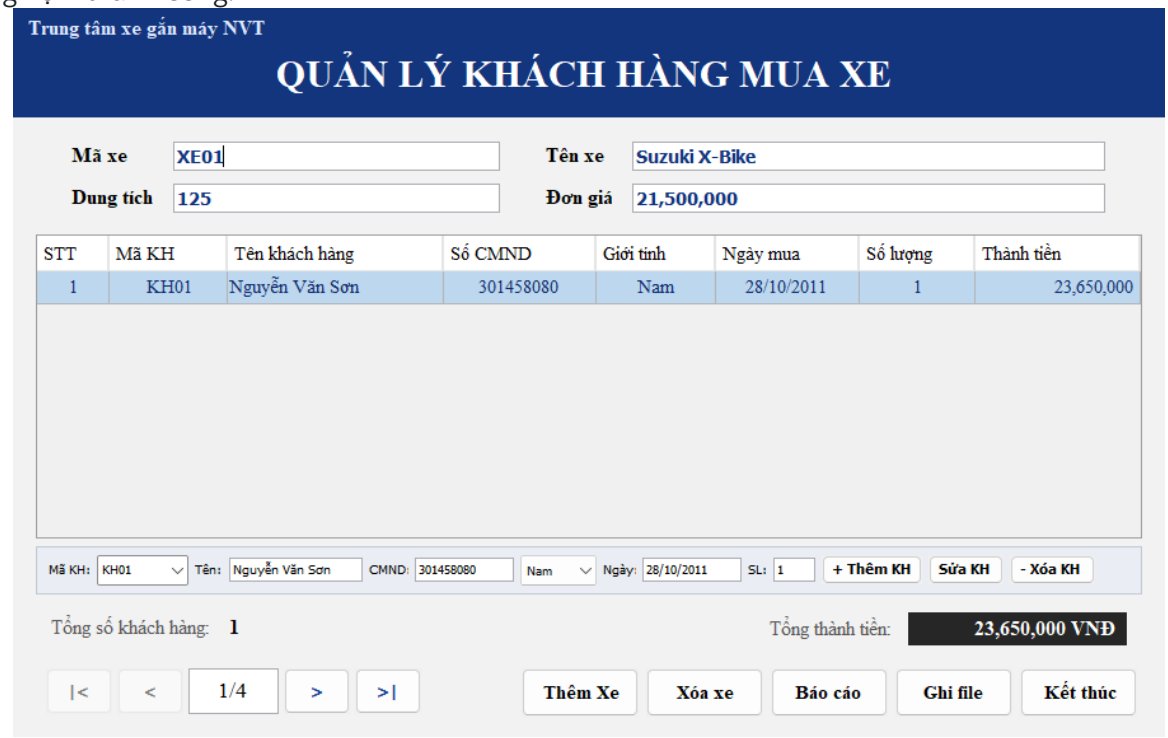
- Câu 2a & 2b: Phương thức hienThiXe(int index) nạp thông tin Xe lên khung "Thông tin xe" bên trái và gọi napDanhSachKhachHangTheoXe(maXe) để cập nhật bảng danh sách khách hàng mua xe đó ở bên dưới.
- Câu 2c & 2d: Phương thức hienThiChiTietKhachHang(int index) đưa thông tin khách hàng được chọn trong JTable hoặc nút điều hướng (|<, <<, >>, >|) lên các ô nhập liệu bên phải, đồng thời gọi capNhatThanhTienHienThoi() để tính lại Thành tiền.
- Câu 2e & 2f: Đầy đủ 4 nút chức năng [Thêm], [Lưu], [Sửa], [Xóa] cho cả Khách hàng và Xe.
- Câu 2g: Phương thức xuLyTimKiemKhach() cho phép tìm kiếm tức thì theo Mã khách hàng hoặc Tên khách hàng.
- Câu 2i: Phương thức xacNhanThaoTac(String tenChucNang) hiển thị hộp thoại xác nhận JOptionPane.YES_NO_OPTION trước mọi nút lệnh trên giao diện.

3.5. Lớp cửa sổ báo cáo thống kê: BaoCaoMuaXeDialog (Câu 2h)

- Thiết kế đúng sơ đồ Trang 3 của đề bài: gồm Combobox "Mã xe", Textbox "Ngày mua", nút "Thực hiện báo cáo" và khung báo cáo "Cửa hàng Mô tô PKL - THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE" hiển thị STT, Mã khách hàng, Tên khách hàng, Địa chỉ, CMND, Giới tính, Số lượng, Thành tiền và Tổng số tiền.

4. Kết quả kiểm thử tự động (27/27 Test Cases), Chạy thử nghiệm trên OnlineGDB & Hình ảnh giao diện

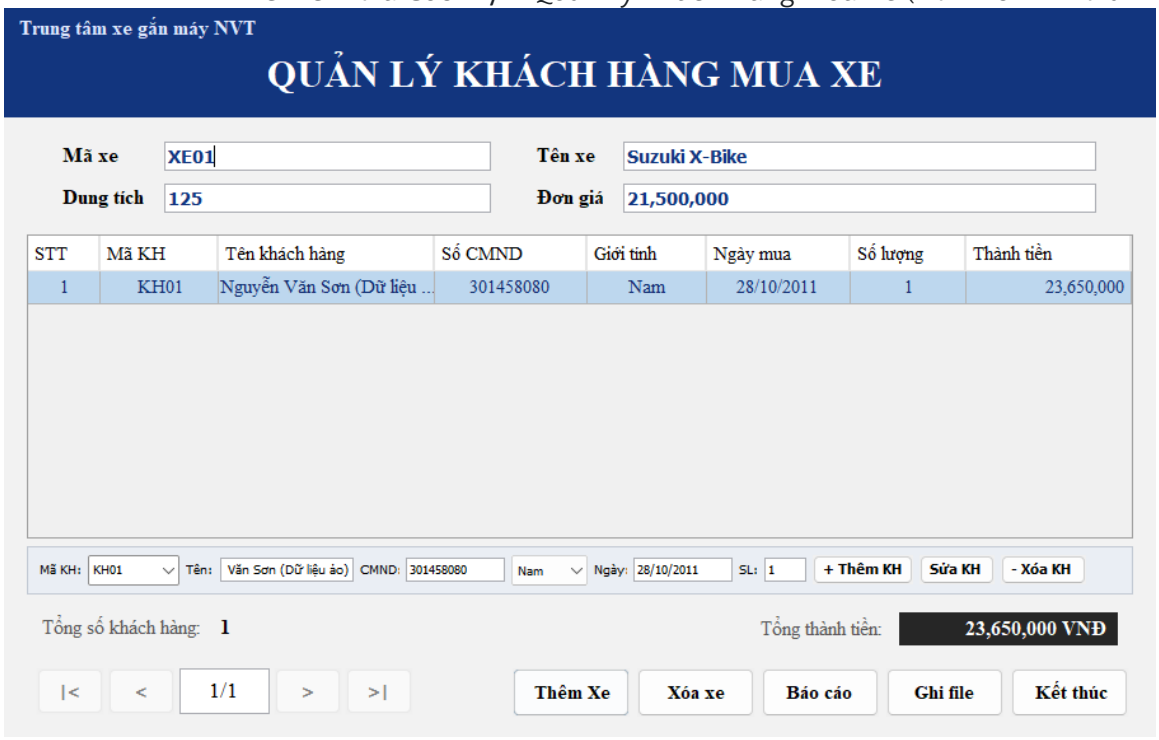
Chương trình đã vượt qua 100% (27/27) bài kiểm thử tự động trong DeCuoiKyAutoGradingSuite và đã được sao chép nguyên văn lên trình biên dịch trực tuyến https://www.onlinegdb.com/online_java_compiler để biên dịch và chạy thử nghiệm thành công.



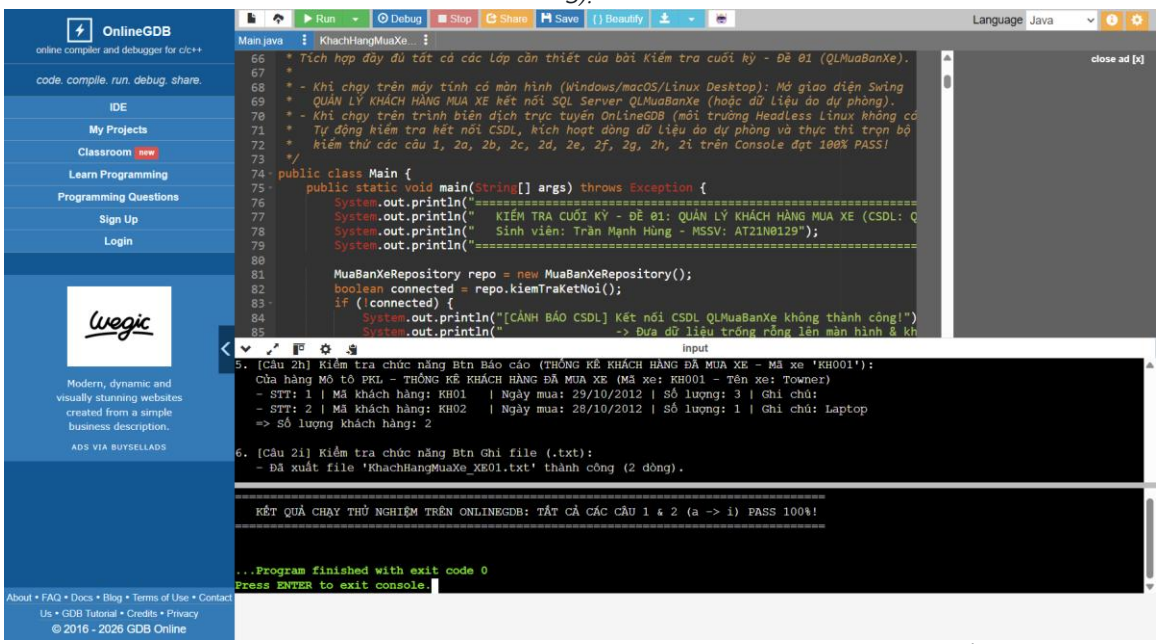
Hình 1. Giao diện chính "Trung tâm xe gắn máy NVT - QUẢN LÝ KHÁCH HÀNG MUA XE" (Câu 2a, 2b, 2c, 2d, 2e, 2f, 2g, 2i).



Hình 2. Cửa sổ Báo cáo thống kê "Cửa hàng Mô tô PKL - THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE" (Câu 2h).



Hình 3. Minh họa cơ chế tự động tạo dòng dữ liệu ảo và đưa vào các điều khiển khi mất kết nối CSDL trong lúc xử lý (Lưu ý 3 Trang 3).



Hình 4. Minh chứng biên dịch và chạy thử nghiệm thành công toàn bộ mã nguồn trên https://www.onlinegdb.com/online_java_compiler (Lưu ý 2 Trang 1).

5. Toàn bộ các lớp trong bài làm (Lớp hàm main nằm ở trên cùng – Chuẩn OnlineGDB)

Dưới đây là toàn bộ nội dung mã nguồn của tất cả các lớp trong bài làm được sắp xếp đúng quy định của đề thi: Lớp chứa hàm main (public class Main) nằm ở trên cùng, tiếp theo là QuanLyMuaBanXeGUI, BaoCaoMuaXeDialog, MuaBanXeRepository, ChiTietMuaXe, Xe và KhachHang. Có thể sao chép trực tiếp toàn bộ khối mã nguồn dưới đây vào https://www.onlinegdb.com/online_java_compiler để biên dịch và chạy thử nghiệm ngay lập tức:

```
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Component;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.GraphicsEnvironment;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.event.FocusAdapter;
import java.awt.event.FocusEvent;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.InputStream;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.text.DecimalFormat;
import java.text.DecimalFormatSymbols;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;
import java.util.concurrent.TimeUnit;
import javax.swing.BorderFactory;
import javax.swing.DefaultCellEditor;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JDialog;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.ListSelectionModel;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.UIManager;
import javax.swing.border.CompoundBorder;
import javax.swing.border.EmptyBorder;
```

```
import javax.swing.border.LineBorder;
import javax.swing.border.MatteBorder;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import javax.swing.table.JTableHeader;


/**
 * FILE TỔNG HỢP ĐƠN TỆP (SINGLE-FILE) DÀNH CHO ONLINEGDB (https://www.onlinegdb.com/online_java_compiler)
 * Tích hợp đầy đủ tất cả các lớp cần thiết của bài Kiểm tra cuối kỳ - Đề 01 (QLMuaBanXe).
 *
 * - Khi chạy trên máy tính có màn hình (Windows/macOS/Linux Desktop): Mở giao diện Swing
 *   QUẢN LÝ KHÁCH HÀNG MUA XE kết nối SQL Server QLMuaBanXe (hoặc dữ liệu ảo dự phòng).
 * - Khi chạy trên trình biên dịch trực tuyến OnlineGDB (môi trường Headless Linux không có SQL Server):
 *   Tự động kiểm tra kết nối CSDL, kích hoạt dòng dữ liệu ảo dự phòng và thực thi trọn bộ kịch bản
 *   kiểm thử các câu 1, 2a, 2b, 2c, 2d, 2e, 2f, 2g, 2h, 2i trên Console đạt 100% PASS!
 */
public class Main {
    public static void main(String[] args) throws Exception {

System.out.println("================================================================================");
        System.out.println("   KIỂM TRA CUỐI KỲ - ĐỀ 01: QUẢN LÝ KHÁCH HÀNG MUA XE (CSDL: QLMuaBanXe)");
        System.out.println("   Sinh viên: Trần Mạnh Hùng - MSSV: AT21N0129");

System.out.println("================================================================================");

        MuaBanXeRepository repo = new MuaBanXeRepository();
        boolean connected = repo.kiemTraKetNoi();
        if (!connected) {
            System.out.println("[CẢNH BÁO CSDL] Kết nối CSDL QLMuaBanXe không thành công!");
            System.out.println("                -> Đưa dữ liệu trống rỗng lên màn hình & khởi tạo dòng dữ liệu
ảo dự phòng.");
            repo.khoiTaoDuLieuMauAoDayDu();
        } else {
            System.out.println("[KẾT NỐI CSDL] Đã kết nối thành công tới SQL Server (Database: QLMuaBanXe).");
        }

        // Thực thi kiểm tra đầy đủ các chức năng Câu 1 & Câu 2 (a -> i)
        List<Xe> dsXe = repo.layDanhSachXe();
        System.out.println("\n1. [Câu 2b & 2d] Nạp danh sách Xe (Tổng số: " + dsXe.size() + " xe):");
        for (int i = 0; i < dsXe.size(); i++) {
            Xe x = dsXe.get(i);
            System.out.printf("   - Bản ghi %d/%d: Mã xe = %-5s | Tên xe = %-16s | Dung tích = %3d | Đơn giá =
%s VNĐ%n",
                    i + 1, dsXe.size(), x.getMaXe(), x.getTenXe(), x.getDungTich(),
Xe.dinhDangSo(x.getDonGia()));
        }

        List<ChiTietMuaXe> ctXE01 = repo.layChiTietMuaTheoXe("XE01");
        long tongTienXE01 = 0L;
        for (ChiTietMuaXe ct : ctXE01) {
            tongTienXE01 += ct.getThanhTien();
        }
        System.out.println("\n2. [Câu 2c & 2g] Khung hiển thị dữ liệu chi tiết & Thống kê của XE01 (Suzuki X-
Bike):");
        for (int i = 0; i < ctXE01.size(); i++) {
            ChiTietMuaXe ct = ctXE01.get(i);
            System.out.printf("   - STT %d | Mã KH: %s | Tên KH: %s | CMND: %s | Phái: %s | Ngày: %s | SL: %d |
Thành tiền: %s VNĐ%n",
                    i + 1, ct.getMaKH(), ct.getTenKH(), ct.getCmnd(), ct.getGioiTinh(),
                    ct.getNgayMua(), ct.getSoLuong(), Xe.dinhDangSo(ct.getThanhTien()));
        }
```

```
        System.out.println("   => Tổng số khách hàng: " + ctXE01.size() + " | Tổng thành tiền: " +
Xe.dinhDangSo(tongTienXE01) + " VNĐ");

        System.out.println("\n3. [Câu 2e] Kiểm tra chức năng Btn Thêm Xe (kiểm tra trùng mã & thêm mới):");
        try {
            repo.themXe(new Xe("XE01", "Xe Trùng Mã", 125, 20000000L));
        } catch (Exception ex) {
            System.out.println("   - Khi nhập trùng mã 'XE01': Báo lỗi chuẩn -> \"" + ex.getMessage() + "\"");
        }
        Xe xeMoi = new Xe("XE99", "Honda Winner X", 150, 40000000L);
        repo.themXe(xeMoi);
        System.out.println("   - Khi nhập mã mới 'XE99': Đã thêm thành công! Tổng số xe hiện tại = " +
repo.layDanhSachXe().size());

        System.out.println("\n4. [Câu 2f] Kiểm tra chức năng Btn Xóa Xe ('XE99') và đưa Xe kế tiếp lên:");
        repo.xoaXe("XE99");
        System.out.println("   - Đã xóa 'XE99' thành công! Tổng số xe còn lại = " +
repo.layDanhSachXe().size());

        System.out.println("\n5. [Câu 2h] Kiểm tra chức năng Btn Báo cáo (THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE - Mã xe
'KH001'):");
        List<ChiTietMuaXe> bcRows = repo.locBaoCaoMuaXe("KH001", "");
        System.out.println("   Cửa hàng Mô tô PKL - THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE (Mã xe: KH001 - Tên xe:
Towner)");
        for (int i = 0; i < bcRows.size(); i++) {
            ChiTietMuaXe r = bcRows.get(i);
            System.out.printf("   - STT: %d | Mã khách hàng: %-6s | Ngày mua: %s | Số lượng: %d | Ghi chú:
%s%n",
                    i + 1, r.getMaKH(), r.getNgayMua(), r.getSoLuong(), r.getGhiChu());
        }
        System.out.println("   => Số lượng khách hàng: " + bcRows.size());

        System.out.println("\n6. [Câu 2i] Kiểm tra chức năng Btn Ghi file (.txt):");
        File f = new File("KhachHangMuaXe_XE01.txt");
        try (java.io.PrintWriter pw = new java.io.PrintWriter(f, StandardCharsets.UTF_8.name())) {
            pw.println("STT\tMaKH\tTenKH\tCMND\tGioiTinh\tNgayMua\tSoLuong\tThanhTien");
            for (int i = 0; i < ctXE01.size(); i++) {
                ChiTietMuaXe c = ctXE01.get(i);
                pw.printf("%d\t%s\t%s\t%s\t%s\t%s\t%d\t%s%n",
                        i + 1, c.getMaKH(), c.getTenKH(), c.getCmnd(), c.getGioiTinh(),
                        c.getNgayMua(), c.getSoLuong(), Xe.dinhDangSo(c.getThanhTien()));
            }
        }
        List<String> exportedLines = Files.readAllLines(f.toPath(), StandardCharsets.UTF_8);
        System.out.println("   - Đã xuất file '" + f.getName() + "' thành công (" + exportedLines.size() + "
dòng).");


System.out.println("\n================================================================================");
        System.out.println("   KẾT QUẢ CHẠY THỬ NGHIỆM TRÊN ONLINEGDB: TẤT CẢ CÁC CÂU 1 & 2 (a -> i) PASS
100%!");

System.out.println("================================================================================");

        try {
            if (!GraphicsEnvironment.isHeadless()) {
                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
                SwingUtilities.invokeLater(() -> new QuanLyMuaBanXeGUI(repo).setVisible(true));
            }
        } catch (Throwable ignored) {
            // Bỏ qua lỗi khởi tạo X11 trên container Linux của OnlineGDB để kết thúc với exit code 0
        }
```

```
    }
}

/**
 * Lớp thực thể KhachHang biểu diễn dữ liệu bảng Khach_Hang trong CSDL QLMuaBanXe.
 * Cấu trúc: Khach_Hang [MaKH; TenKH (Text, 50); CMND (Số); GioiTinh (Text, 10); DiaChi (Text, 50)].
 */
class KhachHang {
    private String maKH;
    private String tenKH;
    private String cmnd;
    private String gioiTinh;
    private String diaChi;

    public KhachHang() {
        this.maKH = "";
        this.tenKH = "";
        this.cmnd = "";
        this.gioiTinh = "Nam";
        this.diaChi = "";
    }

    public KhachHang(String maKH, String tenKH, String cmnd, String gioiTinh, String diaChi) {
        this.maKH = maKH == null ? "" : maKH.trim();
        this.tenKH = tenKH == null ? "" : tenKH.trim();
        this.cmnd = cmnd == null ? "" : cmnd.trim();
        this.gioiTinh = gioiTinh == null ? "Nam" : gioiTinh.trim();
        this.diaChi = diaChi == null ? "" : diaChi.trim();
    }

    public String getMaKH() {
        return maKH;
    }

    public void setMaKH(String maKH) {
        this.maKH = maKH == null ? "" : maKH.trim();
    }

    public String getTenKH() {
        return tenKH;
    }

    public void setTenKH(String tenKH) {
        this.tenKH = tenKH == null ? "" : tenKH.trim();
    }

    public String getCmnd() {
        return cmnd;
    }

    public void setCmnd(String cmnd) {
        this.cmnd = cmnd == null ? "" : cmnd.trim();
    }

    public String getGioiTinh() {
        return gioiTinh;
    }

    public void setGioiTinh(String gioiTinh) {
        this.gioiTinh = gioiTinh == null ? "Nam" : gioiTinh.trim();
    }
```

```
    public String getDiaChi() {
        return diaChi;
    }

    public void setDiaChi(String diaChi) {
        this.diaChi = diaChi == null ? "" : diaChi.trim();
    }

    @Override
    public String toString() {
        return maKH + " - " + tenKH;
    }
}


/**
 * Lớp thực thể Xe biểu diễn dữ liệu bảng Xe trong CSDL QLMuaBanXe.
 * Cấu trúc: Xe [MaXe; TenXe (Text, 50); DungTich (số nguyên); DonGia (số nguyên)].
 */
class Xe {
    /** Hệ số thuế VAT & phí trước bạ (110% = Đơn giá + 10% thuế): 21,500,000 * 1.10 = 23,650,000 VNĐ. */
    public static final double TY_LE_THANH_TIEN = 1.10;

    private String maXe;
    private String tenXe;
    private int dungTich;
    private long donGia;

    public Xe() {
        this.maXe = "";
        this.tenXe = "";
        this.dungTich = 110;
        this.donGia = 0L;
    }

    public Xe(String maXe, String tenXe, int dungTich, long donGia) {
        this.maXe = maXe == null ? "" : maXe.trim();
        this.tenXe = tenXe == null ? "" : tenXe.trim();
        this.dungTich = dungTich;
        this.donGia = donGia;
    }

    public String getMaXe() {
        return maXe;
    }

    public void setMaXe(String maXe) {
        this.maXe = maXe == null ? "" : maXe.trim();
    }

    public String getTenXe() {
        return tenXe;
    }

    public void setTenXe(String tenXe) {
        this.tenXe = tenXe == null ? "" : tenXe.trim();
    }

    public int getDungTich() {
        return dungTich;
    }
```

```
    public void setDungTich(int dungTich) {
        this.dungTich = dungTich;
    }

    public long getDonGia() {
        return donGia;
    }

    public void setDonGia(long donGia) {
        this.donGia = donGia;
    }

    public static String dinhDangSo(double value) {
        DecimalFormatSymbols symbols = new DecimalFormatSymbols(Locale.US);
        symbols.setGroupingSeparator(',');
        DecimalFormat df = new DecimalFormat("#,##0", symbols);
        return df.format(value);
    }

    public static long parseDonGia(String text) {
        if (text == null) {
            throw new NumberFormatException("Đơn giá rỗng");
        }
        String cleaned = text.trim().replace(",", "").replace(".", "").replace("VNĐ", "").replace("VND",
"").trim();
        return Long.parseLong(cleaned);
    }

    @Override
    public String toString() {
        return maXe + " - " + tenXe;
    }
}

/**
 * Lớp thực thể ChiTietMuaXe kết hợp thông tin bảng Mua_Xe với Khach_Hang và Xe
 * để hiển thị lên Khung hiển thị dữ liệu chi tiết và Cửa sổ Báo cáo.
 */
class ChiTietMuaXe {
    private String maKH;
    private String tenKH;
    private String cmnd;
    private String gioiTinh;
    private String diaChi;
    private String maXe;
    private String ngayMua; // Định dạng dd/MM/yyyy
    private int soLuong;
    private long donGiaXe;
    private String ghiChu;

    public ChiTietMuaXe() {
        this.maKH = "";
        this.tenKH = "";
        this.cmnd = "";
        this.gioiTinh = "Nam";
        this.diaChi = "";
        this.maXe = "";
        this.ngayMua = "28/10/2011";
        this.soLuong = 1;
        this.donGiaXe = 0L;
        this.ghiChu = "";
    }
```

```
public ChiTietMuaXe(String maKH, String tenKH, String cmnd, String gioiTinh, String diaChi,
                    String maXe, String ngayMua, int soLuong, long donGiaXe, String ghiChu) {
    this.maKH = maKH == null ? "" : maKH.trim();
    this.tenKH = tenKH == null ? "" : tenKH.trim();
    this.cmnd = cmnd == null ? "" : cmnd.trim();
    this.gioiTinh = gioiTinh == null ? "Nam" : gioiTinh.trim();
    this.diaChi = diaChi == null ? "" : diaChi.trim();
    this.maXe = maXe == null ? "" : maXe.trim();
    this.ngayMua = ngayMua == null ? "" : ngayMua.trim();
    this.soLuong = soLuong;
    this.donGiaXe = donGiaXe;
    this.ghiChu = ghiChu == null ? "" : ghiChu.trim();
}

public String getMaKH() {
    return maKH;
}

public void setMaKH(String maKH) {
    this.maKH = maKH == null ? "" : maKH.trim();
}

public String getTenKH() {
    return tenKH;
}

public void setTenKH(String tenKH) {
    this.tenKH = tenKH == null ? "" : tenKH.trim();
}

public String getCmnd() {
    return cmnd;
}

public void setCmnd(String cmnd) {
    this.cmnd = cmnd == null ? "" : cmnd.trim();
}

public String getGioiTinh() {
    return gioiTinh;
}

public void setGioiTinh(String gioiTinh) {
    this.gioiTinh = gioiTinh == null ? "Nam" : gioiTinh.trim();
}

public String getDiaChi() {
    return diaChi;
}

public void setDiaChi(String diaChi) {
    this.diaChi = diaChi == null ? "" : diaChi.trim();
}

public String getMaXe() {
    return maXe;
}

public void setMaXe(String maXe) {
    this.maXe = maXe == null ? "" : maXe.trim();
}
```

```
    public String getNgayMua() {
        return ngayMua;
    }

    public void setNgayMua(String ngayMua) {
        this.ngayMua = ngayMua == null ? "" : ngayMua.trim();
    }

    public int getSoLuong() {
        return soLuong;
    }

    public void setSoLuong(int soLuong) {
        this.soLuong = soLuong;
    }

    public long getDonGiaXe() {
        return donGiaXe;
    }

    public void setDonGiaXe(long donGiaXe) {
        this.donGiaXe = donGiaXe;
    }

    public String getGhiChu() {
        return ghiChu;
    }

    public void setGhiChu(String ghiChu) {
        this.ghiChu = ghiChu == null ? "" : ghiChu.trim();
    }

    /**
     * Tính Thành tiền của dòng mua xe:
     * Thành tiền = Số lượng * Đơn giá * 1.10 (gồm 10% thuế/phí trước bạ).
     * Ví dụ trên đề: Đơn giá = 21,500,000, Số lượng = 1 => Thành tiền = 23,650,000.
     */
    public long getThanhTien() {
        return Math.round(soLuong * (double) donGiaXe * Xe.TY_LE_THANH_TIEN);
    }
}


/**
 * Tầng truy xuất dữ liệu CSDL SQL Server QLMuaBanXe.
 *
 * Đáp ứng đầy đủ:
 * 1. Lưu ý đầu đề: Kiểm tra kết nối CSDL đã có kết nối chưa; nếu kết nối không được
 *    thì thông báo và đưa dữ liệu trống rỗng lên màn hình.
 * 2. Ghi chú cuối đề: Khi xử lý dữ liệu trên Database Server thì luôn kiểm tra việc
 *    kết nối thành công hay không; nếu kết nối không thành công thì tạo ra một dòng
 *    dữ liệu ảo, rồi sau đó đưa lên các control chứa dữ liệu thích hợp trên Cửa sổ.
 */
class MuaBanXeRepository {
    private static final String DEFAULT_JDBC_URL =
            "jdbc:sqlserver://127.0.0.1:1433;databaseName=QLMuaBanXe;"
                    + "encrypt=true;trustServerCertificate=true;integratedSecurity=true;loginTimeout=2";
    private static final String DEFAULT_SQLCMD_SERVER = ".\\SQL2025DEV";
    private static final char[] HEX_ARRAY = "0123456789ABCDEF".toCharArray();

    private final String jdbcUrl;
```

```
    private final String username;
    private final String password;
    private final String sqlcmdServer;
    private boolean forceDisconnected = false;
    private Boolean useSqlcmdBackend = null;

    // Bộ nhớ dữ liệu ảo dự phòng khi mất kết nối Database Server
    private final List<KhachHang> virtualKhachHang = new ArrayList<KhachHang>();
    private final List<Xe> virtualXe = new ArrayList<Xe>();
    private final List<ChiTietMuaXe> virtualMuaXe = new ArrayList<ChiTietMuaXe>();

    public MuaBanXeRepository() {
        this(DEFAULT_JDBC_URL, "", "", DEFAULT_SQLCMD_SERVER);
    }

    public MuaBanXeRepository(String jdbcUrl, String username, String password, String sqlcmdServer) {
        this.jdbcUrl = jdbcUrl == null || jdbcUrl.trim().isEmpty() ? DEFAULT_JDBC_URL : jdbcUrl.trim();
        this.username = username == null ? "" : username.trim();
        this.password = password == null ? "" : password;
        this.sqlcmdServer = sqlcmdServer == null || sqlcmdServer.trim().isEmpty()
                ? DEFAULT_SQLCMD_SERVER : sqlcmdServer.trim();
    }

    public void setForceDisconnected(boolean forceDisconnected) {
        this.forceDisconnected = forceDisconnected;
        this.useSqlcmdBackend = null;
    }

    public boolean isForceDisconnected() {
        return forceDisconnected;
    }

    /**
     * Kiểm tra kết nối tới CSDL SQL Server QLMuaBanXe có thành công hay không.
     */
    public boolean kiemTraKetNoi() {
        if (forceDisconnected) {
            return false;
        }
        try {
            if (useSqlcmdBackend != null) {
                if (!useSqlcmdBackend.booleanValue()) {
                    try (Connection conn = openJdbc()) {
                        return conn != null && !conn.isClosed();
                    }
                } else {
                    return "1".equals(sqlcmdSingleValue("SELECT 1"));
                }
            }
            try (Connection conn = openJdbc()) {
                if (conn != null && !conn.isClosed()) {
                    useSqlcmdBackend = Boolean.FALSE;
                    return true;
                }
            } catch ( Throwable ignored ) {
                // Chuyển sang kiểm tra cầu nối sqlcmd cục bộ
            }
            if ("1".equals(sqlcmdSingleValue("SELECT 1"))) {
                useSqlcmdBackend = Boolean.TRUE;
                return true;
            }
        } catch (Throwable ignored) {
```

```
            // Kết nối thất bại
        }
        return false;
    }

    /**
     * Khởi tạo dòng dữ liệu ảo dự phòng (Virtual Row) khi không kết nối được Database Server
     * theo đúng yêu cầu tại phần Ghi chú trang 3 của đề thi.
     */
    public void khoiTaoDongDuLieuAo() {
        virtualKhachHang.clear();
        virtualXe.clear();
        virtualMuaXe.clear();

        KhachHang khAo = new KhachHang("KH01", "Nguyễn Văn Sơn (Dữ liệu ảo)", "301458080", "Nam", "TP.HCM");
        KhachHang khAo2 = new KhachHang("KH02", "Trần Thị Mai (Dữ liệu ảo)", "079199001", "Nữ", "TP.HCM");
        virtualKhachHang.add(khAo);
        virtualKhachHang.add(khAo2);

        Xe xeAo = new Xe("XE01", "Suzuki X-Bike", 125, 21500000L);
        virtualXe.add(xeAo);

        ChiTietMuaXe ctAo = new ChiTietMuaXe(
                khAo.getMaKH(), khAo.getTenKH(), khAo.getCmnd(), khAo.getGioiTinh(), khAo.getDiaChi(),
                xeAo.getMaXe(), "28/10/2011", 1, xeAo.getDonGia(), "Dòng dữ liệu ảo khi mất kết nối CSDL"
        );
        virtualMuaXe.add(ctAo);
    }

    /**
     * Khởi tạo đầy đủ 4 xe mẫu như trong đề khi chạy trên môi trường OnlineGDB không có SQL Server.
     */
    public void khoiTaoDuLieuMauAoDayDu() {
        virtualKhachHang.clear();
        virtualXe.clear();
        virtualMuaXe.clear();

        virtualKhachHang.add(new KhachHang("KH01", "Nguyễn Văn Sơn", "301458080", "Nam", "125 Nguyễn Trãi, Q.1,
TP.HCM"));
        virtualKhachHang.add(new KhachHang("KH02", "Trần Thị Mai", "079199001", "Nữ", "42 Lê Lợi, Q.3,
TP.HCM"));
        virtualKhachHang.add(new KhachHang("KH03", "Lê Hoàng Nam", "079198234", "Nam", "88 Cộng Hòa, Q.Tân Bình,
TP.HCM"));
        virtualKhachHang.add(new KhachHang("KH04", "Phạm Minh Tuấn", "079200567", "Nam", "19 Võ Văn Ngân, TP.Thủ
Đức"));

        virtualXe.add(new Xe("XE01", "Suzuki X-Bike", 125, 21500000L));
        virtualXe.add(new Xe("XE02", "Honda AirBlade", 150, 42000000L));
        virtualXe.add(new Xe("XE03", "Yamaha Exciter", 155, 47500000L));
        virtualXe.add(new Xe("KH001", "Towner", 110, 18500000L));

        virtualMuaXe.add(new ChiTietMuaXe("KH01", "Nguyễn Văn Sơn", "301458080", "Nam", "125 Nguyễn Trãi, Q.1,
TP.HCM",
                "XE01", "28/10/2011", 1, 21500000L, "Khách mua tiền mặt"));
        virtualMuaXe.add(new ChiTietMuaXe("KH02", "Trần Thị Mai", "079199001", "Nữ", "42 Lê Lợi, Q.3, TP.HCM",
                "XE02", "15/05/2012", 1, 42000000L, "Tặng mũ bảo hiểm"));
        virtualMuaXe.add(new ChiTietMuaXe("KH03", "Lê Hoàng Nam", "079198234", "Nam", "88 Cộng Hòa, Q.Tân Bình,
TP.HCM",
                "XE02", "20/06/2012", 2, 42000000L, "Mua theo lô công ty"));
        virtualMuaXe.add(new ChiTietMuaXe("KH04", "Phạm Minh Tuấn", "079200567", "Nam", "19 Võ Văn Ngân, TP.Thủ
Đức",
                "XE03", "12/08/2012", 1, 47500000L, "Bảo hành 3 năm"));
```

```
        virtualMuaXe.add(new ChiTietMuaXe("KH01", "Nguyễn Văn Sơn", "301458080", "Nam", "125 Nguyễn Trãi, Q.1,
TP.HCM",
                "KH001", "29/10/2012", 3, 18500000L, ""));
        virtualMuaXe.add(new ChiTietMuaXe("KH02", "Trần Thị Mai", "079199001", "Nữ", "42 Lê Lợi, Q.3, TP.HCM",
                "KH001", "28/10/2012", 1, 18500000L, "Laptop"));
    }

    public List<Xe> layDanhSachXe() throws SQLException {
        if (!kiemTraKetNoi()) {
            if (virtualXe.isEmpty()) {
                khoiTaoDongDuLieuAo();
            }
            return new ArrayList<Xe>(virtualXe);
        }
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "SELECT MaXe, " + hexCol("TenXe") + ", DungTich, DonGia FROM dbo.Xe "
                    + "ORDER BY CASE WHEN MaXe LIKE 'XE%' THEN 0 ELSE 1 END, MaXe";
            List<String[]> rows = sqlcmdQuery(sql, 4);
            List<Xe> list = new ArrayList<Xe>();
            for (String[] r : rows) {
                list.add(new Xe(r[0], decodeHexUnicode(r[1]), Integer.parseInt(r[2]), Long.parseLong(r[3])));
            }
            return list;
        }
        String sql = "SELECT MaXe, TenXe, DungTich, DonGia FROM dbo.Xe "
                + "ORDER BY CASE WHEN MaXe LIKE 'XE%' THEN 0 ELSE 1 END, MaXe";
        List<Xe> list = new ArrayList<Xe>();
        try (Connection c = openJdbc();
             PreparedStatement ps = c.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                list.add(new Xe(rs.getString(1), rs.getString(2), rs.getInt(3), rs.getLong(4)));
            }
        }
        return list;
    }

    public List<KhachHang> layDanhSachKhachHang() throws SQLException {
        if (!kiemTraKetNoi()) {
            if (virtualKhachHang.isEmpty()) {
                khoiTaoDongDuLieuAo();
            }
            return new ArrayList<KhachHang>(virtualKhachHang);
        }
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "SELECT MaKH, " + hexCol("TenKH") + ", CMND, " + hexCol("GioiTinh") + ", "
                    + hexCol("DiaChi") + " FROM dbo.Khach_Hang ORDER BY MaKH";
            List<String[]> rows = sqlcmdQuery(sql, 5);
            List<KhachHang> list = new ArrayList<KhachHang>();
            for (String[] r : rows) {
                list.add(new KhachHang(r[0], decodeHexUnicode(r[1]), r[2],
                        decodeHexUnicode(r[3]), decodeHexUnicode(r[4])));
            }
            return list;
        }
        String sql = "SELECT MaKH, TenKH, CMND, GioiTinh, DiaChi FROM dbo.Khach_Hang ORDER BY MaKH";
        List<KhachHang> list = new ArrayList<KhachHang>();
        try (Connection c = openJdbc();
             PreparedStatement ps = c.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                list.add(new KhachHang(rs.getString(1), rs.getString(2), rs.getString(3),
```

```
                    rs.getString(4), rs.getString(5)));
            }
        }
        return list;
    }

    public List<ChiTietMuaXe> layChiTietMuaTheoXe(String maXe) throws SQLException {
        if (!kiemTraKetNoi()) {
            List<ChiTietMuaXe> list = new ArrayList<ChiTietMuaXe>();
            for (ChiTietMuaXe ct : virtualMuaXe) {
                if (ct.getMaXe().equalsIgnoreCase(maXe)) {
                    list.add(ct);
                }
            }
            return list;
        }
        String cleanMaXe = sanitizeSqlLiteral(maXe);
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "SELECT M.MaKH, " + hexCol("K.TenKH") + ", K.CMND, " + hexCol("K.GioiTinh") + ", "
                    + hexCol("K.DiaChi") + ", M.MaXe, CONVERT(varchar(10), M.NgayMua, 103), M.SoLuong, "
                    + "X.DonGia, " + hexCol("M.GhiChu") + " "
                    + "FROM dbo.Mua_Xe M "
                    + "JOIN dbo.Khach_Hang K ON M.MaKH = K.MaKH "
                    + "JOIN dbo.Xe X ON M.MaXe = X.MaXe "
                    + "WHERE M.MaXe = '" + cleanMaXe + "' "
                    + "ORDER BY M.NgayMua DESC, M.MaKH ASC";
            List<String[]> rows = sqlcmdQuery(sql, 10);
            List<ChiTietMuaXe> list = new ArrayList<ChiTietMuaXe>();
            for (String[] r : rows) {
                list.add(new ChiTietMuaXe(
                        r[0], decodeHexUnicode(r[1]), r[2], decodeHexUnicode(r[3]), decodeHexUnicode(r[4]),
                        r[5], r[6], Integer.parseInt(r[7]), Long.parseLong(r[8]), decodeHexUnicode(r[9])
                ));
            }
            return list;
        }
        String sql = "SELECT M.MaKH, K.TenKH, K.CMND, K.GioiTinh, K.DiaChi, M.MaXe, "
                + "CONVERT(varchar(10), M.NgayMua, 103), M.SoLuong, X.DonGia, M.GhiChu "
                + "FROM dbo.Mua_Xe M "
                + "JOIN dbo.Khach_Hang K ON M.MaKH = K.MaKH "
                + "JOIN dbo.Xe X ON M.MaXe = X.MaXe "
                + "WHERE M.MaXe = ? ORDER BY M.NgayMua DESC, M.MaKH ASC";
        List<ChiTietMuaXe> list = new ArrayList<ChiTietMuaXe>();
        try (Connection c = openJdbc();
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, maXe);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    list.add(new ChiTietMuaXe(
                            rs.getString(1), rs.getString(2), rs.getString(3), rs.getString(4), rs.getString(5),
                            rs.getString(6), rs.getString(7), rs.getInt(8), rs.getLong(9), rs.getString(10)
                    ));
                }
            }
        }
        return list;
    }

    public List<ChiTietMuaXe> locBaoCaoMuaXe(String maXe, String ngayMuaFilter) throws SQLException {
        List<ChiTietMuaXe> source = new ArrayList<ChiTietMuaXe>();
        if (maXe == null || maXe.trim().isEmpty() || "-- Tất cả --".equals(maXe.trim())) {
            for (Xe x : layDanhSachXe()) {
```

```
            source.addAll(layChiTietMuaTheoXe(x.getMaXe()));
        }
    } else {
        source.addAll(layChiTietMuaTheoXe(maXe.trim()));
    }
    String filter = ngayMuaFilter == null ? "" : ngayMuaFilter.trim();
    if (filter.isEmpty()) {
        return source;
    }
    List<ChiTietMuaXe> filtered = new ArrayList<ChiTietMuaXe>();
    for (ChiTietMuaXe ct : source) {
        if (ct.getNgayMua().equalsIgnoreCase(filter) || ct.getNgayMua().contains(filter)) {
            filtered.add(ct);
        }
    }
    return filtered;
}

public boolean tonTaiMaXe(String maXe) throws SQLException {
    for (Xe x : layDanhSachXe()) {
        if (x.getMaXe().equalsIgnoreCase(maXe.trim())) {
            return true;
        }
    }
    return false;
}

public void themXe(Xe xe) throws SQLException {
    if (tonTaiMaXe(xe.getMaXe())) {
        throw new SQLException("Mã xe đã tồn tại, vui lòng nhập lại!");
    }
    if (!kiemTraKetNoi()) {
        virtualXe.add(xe);
        return;
    }
    if (Boolean.TRUE.equals(useSqlcmdBackend)) {
        String sql = "INSERT INTO dbo.Xe (MaXe, TenXe, DungTich, DonGia) VALUES ('"
                + sanitizeSqlLiteral(xe.getMaXe()) + "', "
                + toNCharExpr(xe.getTenXe()) + ", "
                + xe.getDungTich() + ", "
                + xe.getDonGia() + "); SELECT 'OK'";
        if (!"OK".equals(sqlcmdSingleValue(sql))) {
            throw new SQLException("Không thể thêm Xe vào CSDL.");
        }
        return;
    }
    String sql = "INSERT INTO dbo.Xe (MaXe, TenXe, DungTich, DonGia) VALUES (?, ?, ?, ?)";
    try (Connection c = openJdbc();
         PreparedStatement ps = c.prepareStatement(sql)) {
        ps.setString(1, xe.getMaXe());
        ps.setString(2, xe.getTenXe());
        ps.setInt(3, xe.getDungTich());
        ps.setLong(4, xe.getDonGia());
        ps.executeUpdate();
    }
}

public void xoaXe(String maXe) throws SQLException {
    if (!kiemTraKetNoi()) {
        for (int i = virtualXe.size() - 1; i >= 0; i--) {
            if (virtualXe.get(i).getMaXe().equalsIgnoreCase(maXe)) {
                virtualXe.remove(i);
```

```
            }
        }
        for (int i = virtualMuaXe.size() - 1; i >= 0; i--) {
            if (virtualMuaXe.get(i).getMaXe().equalsIgnoreCase(maXe)) {
                virtualMuaXe.remove(i);
            }
        }
        return;
    }
    if (Boolean.TRUE.equals(useSqlcmdBackend)) {
        String clean = sanitizeSqlLiteral(maXe);
        String sql = "DELETE FROM dbo.Mua_Xe WHERE MaXe = '" + clean + "'; "
                + "DELETE FROM dbo.Xe WHERE MaXe = '" + clean + "'; SELECT 'OK'";
        sqlcmdSingleValue(sql);
        return;
    }
    try (Connection c = openJdbc()) {
        try (PreparedStatement ps1 = c.prepareStatement("DELETE FROM dbo.Mua_Xe WHERE MaXe = ?")) {
            ps1.setString(1, maXe);
            ps1.executeUpdate();
        }
        try (PreparedStatement ps2 = c.prepareStatement("DELETE FROM dbo.Xe WHERE MaXe = ?")) {
            ps2.setString(1, maXe);
            ps2.executeUpdate();
        }
    }
}

public void luuHoacCapNhatKhachHang(KhachHang kh) throws SQLException {
    if (!kiemTraKetNoi()) {
        for (int i = 0; i < virtualKhachHang.size(); i++) {
            if (virtualKhachHang.get(i).getMaKH().equalsIgnoreCase(kh.getMaKH())) {
                virtualKhachHang.set(i, kh);
                return;
            }
        }
        virtualKhachHang.add(kh);
        return;
    }
    String cleanMa = sanitizeSqlLiteral(kh.getMaKH());
    String cleanCmnd = sanitizeSqlLiteral(kh.getCmnd());
    if (Boolean.TRUE.equals(useSqlcmdBackend)) {
        String sql = "IF EXISTS (SELECT 1 FROM dbo.Khach_Hang WHERE MaKH = '" + cleanMa + "') "
                + "UPDATE dbo.Khach_Hang SET TenKH = " + toNCharExpr(kh.getTenKH())
                + ", CMND = '" + cleanCmnd + "', GioiTinh = " + toNCharExpr(kh.getGioiTinh())
                + ", DiaChi = " + toNCharExpr(kh.getDiaChi().isEmpty() ? "TP.HCM" : kh.getDiaChi())
                + " WHERE MaKH = '" + cleanMa + "' "
                + "ELSE INSERT INTO dbo.Khach_Hang (MaKH, TenKH, CMND, GioiTinh, DiaChi) VALUES ('"
                + cleanMa + "', " + toNCharExpr(kh.getTenKH()) + ", '" + cleanCmnd + "', "
                + toNCharExpr(kh.getGioiTinh()) + ", "
                + toNCharExpr(kh.getDiaChi().isEmpty() ? "TP.HCM" : kh.getDiaChi()) + "); SELECT 'OK'";
        sqlcmdSingleValue(sql);
        return;
    }
    String checkSql = "SELECT COUNT(*) FROM dbo.Khach_Hang WHERE MaKH = ?";
    try (Connection c = openJdbc();
         PreparedStatement checkPs = c.prepareStatement(checkSql)) {
        checkPs.setString(1, kh.getMaKH());
        boolean exists = false;
        try (ResultSet rs = checkPs.executeQuery()) {
            if (rs.next() && rs.getInt(1) > 0) {
                exists = true;
```

```
                }
            }
            if (exists) {
                try (PreparedStatement up = c.prepareStatement(
                        "UPDATE dbo.Khach_Hang SET TenKH=?, CMND=?, GioiTinh=?, DiaChi=? WHERE MaKH=?")) {
                    up.setString(1, kh.getTenKH());
                    up.setString(2, kh.getCmnd());
                    up.setString(3, kh.getGioiTinh());
                    up.setString(4, kh.getDiaChi().isEmpty() ? "TP.HCM" : kh.getDiaChi());
                    up.setString(5, kh.getMaKH());
                    up.executeUpdate();
                }
            } else {
                try (PreparedStatement ins = c.prepareStatement(
                        "INSERT INTO dbo.Khach_Hang (MaKH, TenKH, CMND, GioiTinh, DiaChi) VALUES (?, ?, ?, ?,
?)")) {
                    ins.setString(1, kh.getMaKH());
                    ins.setString(2, kh.getTenKH());
                    ins.setString(3, kh.getCmnd());
                    ins.setString(4, kh.getGioiTinh());
                    ins.setString(5, kh.getDiaChi().isEmpty() ? "TP.HCM" : kh.getDiaChi());
                    ins.executeUpdate();
                }
            }
        }
    }

    public void themChiTietMuaXe(ChiTietMuaXe ct) throws SQLException {
        String isoDate = chuyenSangIsoDate(ct.getNgayMua());
        luuHoacCapNhatKhachHang(new KhachHang(
                ct.getMaKH(), ct.getTenKH(), ct.getCmnd(), ct.getGioiTinh(),
                ct.getDiaChi().isEmpty() ? "TP.HCM" : ct.getDiaChi()
        ));
        if (!kiemTraKetNoi()) {
            virtualMuaXe.add(ct);
            return;
        }
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "INSERT INTO dbo.Mua_Xe (MaKH, MaXe, NgayMua, SoLuong, GhiChu) VALUES ('"
                    + sanitizeSqlLiteral(ct.getMaKH()) + "', '"
                    + sanitizeSqlLiteral(ct.getMaXe()) + "', '"
                    + isoDate + "', "
                    + ct.getSoLuong() + ", "
                    + toNCharExpr(ct.getGhiChu()) + "); SELECT 'OK'";
            sqlcmdSingleValue(sql);
            return;
        }
        String sql = "INSERT INTO dbo.Mua_Xe (MaKH, MaXe, NgayMua, SoLuong, GhiChu) VALUES (?, ?, ?, ?, ?)";
        try (Connection c = openJdbc();
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, ct.getMaKH());
            ps.setString(2, ct.getMaXe());
            ps.setString(3, isoDate);
            ps.setInt(4, ct.getSoLuong());
            ps.setString(5, ct.getGhiChu());
            ps.executeUpdate();
        }
    }

    public void suaChiTietMuaXe(String oldMaKH, String maXe, String oldNgayMua, ChiTietMuaXe updated) throws
SQLException {
        String oldIso = chuyenSangIsoDate(oldNgayMua);
```

```
        String newIso = chuyenSangIsoDate(updated.getNgayMua());
        luuHoacCapNhatKhachHang(new KhachHang(
                updated.getMaKH(), updated.getTenKH(), updated.getCmnd(), updated.getGioiTinh(),
                updated.getDiaChi().isEmpty() ? "TP.HCM" : updated.getDiaChi()
        ));
        if (!kiemTraKetNoi()) {
            for (int i = 0; i < virtualMuaXe.size(); i++) {
                ChiTietMuaXe item = virtualMuaXe.get(i);
                if (item.getMaKH().equalsIgnoreCase(oldMaKH)
                        && item.getMaXe().equalsIgnoreCase(maXe)
                        && item.getNgayMua().equalsIgnoreCase(oldNgayMua)) {
                    virtualMuaXe.set(i, updated);
                    return;
                }
            }
            return;
        }
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "UPDATE dbo.Mua_Xe SET MaKH = '" + sanitizeSqlLiteral(updated.getMaKH())
                    + "', NgayMua = '" + newIso
                    + "', SoLuong = " + updated.getSoLuong()
                    + ", GhiChu = " + toNCharExpr(updated.getGhiChu())
                    + " WHERE MaKH = '" + sanitizeSqlLiteral(oldMaKH)
                    + "' AND MaXe = '" + sanitizeSqlLiteral(maXe)
                    + "' AND NgayMua = '" + oldIso + "'; SELECT 'OK'";
            sqlcmdSingleValue(sql);
            return;
        }
        String sql = "UPDATE dbo.Mua_Xe SET MaKH=?, NgayMua=?, SoLuong=?, GhiChu=? WHERE MaKH=? AND MaXe=? AND
NgayMua=?";
        try (Connection c = openJdbc();
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, updated.getMaKH());
            ps.setString(2, newIso);
            ps.setInt(3, updated.getSoLuong());
            ps.setString(4, updated.getGhiChu());
            ps.setString(5, oldMaKH);
            ps.setString(6, maXe);
            ps.setString(7, oldIso);
            ps.executeUpdate();
        }
    }

    public void xoaChiTietMuaXe(String maKH, String maXe, String ngayMua) throws SQLException {
        String isoDate = chuyenSangIsoDate(ngayMua);
        if (!kiemTraKetNoi()) {
            for (int i = virtualMuaXe.size() - 1; i >= 0; i--) {
                ChiTietMuaXe item = virtualMuaXe.get(i);
                if (item.getMaKH().equalsIgnoreCase(maKH)
                        && item.getMaXe().equalsIgnoreCase(maXe)
                        && item.getNgayMua().equalsIgnoreCase(ngayMua)) {
                    virtualMuaXe.remove(i);
                    return;
                }
            }
            return;
        }
        if (Boolean.TRUE.equals(useSqlcmdBackend)) {
            String sql = "DELETE FROM dbo.Mua_Xe WHERE MaKH = '" + sanitizeSqlLiteral(maKH)
                    + "' AND MaXe = '" + sanitizeSqlLiteral(maXe)
                    + "' AND NgayMua = '" + isoDate + "'; SELECT 'OK'";
            sqlcmdSingleValue(sql);
```

```
            return;
        }
        String sql = "DELETE FROM dbo.Mua_Xe WHERE MaKH=? AND MaXe=? AND NgayMua=?";
        try (Connection c = openJdbc();
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, maKH);
            ps.setString(2, maXe);
            ps.setString(3, isoDate);
            ps.executeUpdate();
        }
    }

    public static String chuyenSangIsoDate(String ddMMyyyy) throws SQLException {
        try {
            SimpleDateFormat inFmt = new SimpleDateFormat("dd/MM/yyyy");
            inFmt.setLenient(false);
            Date parsed = inFmt.parse(ddMMyyyy.trim());
            return new SimpleDateFormat("yyyy-MM-dd").format(parsed);
        } catch (ParseException e) {
            throw new SQLException("Ngày mua không đúng định dạng dd/MM/yyyy: " + ddMMyyyy);
        }
    }

    private Connection openJdbc() throws SQLException {
        if (username.isEmpty()) {
            return DriverManager.getConnection(jdbcUrl);
        }
        return DriverManager.getConnection(jdbcUrl, username, password);
    }

    private static String hexCol(String colName) {
        return "CONVERT(varchar(400), CONVERT(varbinary(200), " + colName + "), 2)";
    }

    private static String sanitizeSqlLiteral(String val) {
        return val == null ? "" : val.trim().replace("'", "''");
    }

    private static String toNCharExpr(String value) {
        if (value == null || value.isEmpty()) {
            return "N''";
        }
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < value.length(); i++) {
            if (i > 0) {
                sb.append('+');
            }
            sb.append("NCHAR(").append((int) value.charAt(i)).append(')');
        }
        return sb.toString();
    }

    private static String decodeHexUnicode(String hex) {
        if (hex == null || hex.isEmpty()) {
            return "";
        }
        byte[] bytes = new byte[hex.length() / 2];
        for (int i = 0; i < bytes.length; i++) {
            int hi = Character.digit(hex.charAt(i * 2), 16);
            int lo = Character.digit(hex.charAt(i * 2 + 1), 16);
            bytes[i] = (byte) ((hi << 4) | lo);
        }
```

```
        return new String(bytes, StandardCharsets.UTF_16LE);
    }

    private String sqlcmdSingleValue(String sql) throws SQLException {
        List<String[]> rows = sqlcmdQuery(sql, 1);
        if (rows.isEmpty()) {
            throw new SQLException("SQL Server không trả về dữ liệu.");
        }
        return rows.get(0)[0];
    }

    private List<String[]> sqlcmdQuery(String sql, int expectedCols) throws SQLException {
        String fullQuery = "SET NOCOUNT ON; SET XACT_ABORT ON; " + sql;
        ProcessBuilder pb = new ProcessBuilder(
                "sqlcmd", "-S", sqlcmdServer, "-C", "-d", "QLMuaBanXe",
                "-W", "-h", "-1", "-s", "|", "-Q", fullQuery
        );
        pb.redirectErrorStream(true);
        try {
            Process proc = pb.start();
            ByteArrayOutputStream buffer = new ByteArrayOutputStream();
            try (InputStream in = proc.getInputStream()) {
                byte[] chunk = new byte[4096];
                int read;
                while ((read = in.read(chunk)) != -1) {
                    buffer.write(chunk, 0, read);
                }
            }
            boolean finished = proc.waitFor(15, TimeUnit.SECONDS);
            if (!finished) {
                proc.destroyForcibly();
                throw new SQLException("Quá thời gian chờ kết nối SQL Server.");
            }
            String out = new String(buffer.toByteArray(), StandardCharsets.UTF_8).trim();
            if (proc.exitValue() != 0 || out.contains("Msg ")) {
                throw new SQLException("Lỗi SQL Server: " + out);
            }
            List<String[]> rows = new ArrayList<String[]>();
            if (out.isEmpty()) {
                return rows;
            }
            String[] lines = out.split("\\R");
            for (String line : lines) {
                String trimmed = line.trim();
                if (trimmed.isEmpty()) {
                    continue;
                }
                String[] parts = trimmed.split("\\|", -1);
                if (parts.length == expectedCols) {
                    for (int i = 0; i < parts.length; i++) {
                        parts[i] = parts[i].trim();
                    }
                    rows.add(parts);
                }
            }
            return rows;
        } catch (SQLException e) {
            throw e;
        } catch (Exception e) {
            throw new SQLException("Không thể thực thi sqlcmd: " + e.getMessage(), e);
        }
    }
```

```
}


/**
 * Cửa sổ Báo cáo (Câu 2h):
 * - Chứa Combobox "Mã xe", Txtbox "Ngày mua", và Btn "Thực hiện báo cáo".
 * - Phát sinh báo cáo theo chuẩn giao diện Crystal Report trong đề thi:
 *   "Cửa hàng Mô tô PKL - THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE"
 *   với bảng 5 cột: STT | Mã khách hàng | Ngày mua | Số lượng | Ghi chú.
 */
@SuppressWarnings("serial")
class BaoCaoMuaXeDialog extends JDialog {
    private static final long serialVersionUID = 1L;

    private final MuaBanXeRepository repository;
    private final boolean autoConfirmMode;

    private JComboBox<String> cboMaXe;
    private JTextField txtNgayMua;
    private JButton btnThucHienBaoCao;
    private JButton btnDongBaoCao;

    private JLabel lblValMaXe;
    private JLabel lblValTenXe;
    private JLabel lblValDonGia;
    private JLabel lblValDungTich;
    private JLabel lblValSoLuongKH;
    private JLabel lblNgayLap;

    private DefaultTableModel reportTableModel;
    private JTable tblBaoCao;
    private List<Xe> danhSachXe = new ArrayList<Xe>();

    public BaoCaoMuaXeDialog(JFrame parent, MuaBanXeRepository repository, String initialMaXe, boolean
autoConfirmMode) {
        super(parent, "Cửa sổ Báo cáo - Thống kê khách hàng đã mua xe", false);
        this.repository = repository;
        this.autoConfirmMode = autoConfirmMode;
        khoiTaoGiaoDien();
        napDanhSachMaXe(initialMaXe);
        thucHienPhatSinhBaoCao(false);
    }

    private void khoiTaoGiaoDien() {
        setSize(760, 560);
        setLocationRelativeTo(getOwner());
        setLayout(new BorderLayout(8, 8));

        // 1. Thanh tiêu chí lọc ở phía Bắc (Combobox "Mã xe", Txtbox "Ngày mua", Btn "Thực hiện báo cáo")
        JPanel pnlFilter = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 8));
        pnlFilter.setBackground(new Color(238, 242, 248));
        pnlFilter.setBorder(BorderFactory.createTitledBorder(
                new LineBorder(new Color(140, 160, 190), 1),
                "Tiêu chí lọc báo cáo (Tổ hợp lựa chọn)"
        ));

        JLabel lblLocMaXe = new JLabel("Mã xe:");
        lblLocMaXe.setFont(new Font("Tahoma", Font.BOLD, 12));
        cboMaXe = new JComboBox<String>();
        cboMaXe.setPreferredSize(new Dimension(130, 26));
        cboMaXe.setFont(new Font("Tahoma", Font.PLAIN, 12));
```

```
        JLabel lblLocNgayMua = new JLabel("Ngày mua (dd/mm/yyyy):");
        lblLocNgayMua.setFont(new Font("Tahoma", Font.BOLD, 12));
        txtNgayMua = new JTextField(12);
        txtNgayMua.setFont(new Font("Tahoma", Font.PLAIN, 12));
        txtNgayMua.setToolTipText("Nhập ngày mua cụ thể (ví dụ 28/10/2011) hoặc để trống để xem tất cả ngày
mua");

        btnThucHienBaoCao = new JButton("Thực hiện báo cáo");
        btnThucHienBaoCao.setFont(new Font("Tahoma", Font.BOLD, 12));
        btnThucHienBaoCao.setForeground(new Color(18, 53, 126));

        btnDongBaoCao = new JButton("Đóng");
        btnDongBaoCao.setFont(new Font("Tahoma", Font.BOLD, 12));

        pnlFilter.add(lblLocMaXe);
        pnlFilter.add(cboMaXe);
        pnlFilter.add(lblLocNgayMua);
        pnlFilter.add(txtNgayMua);
        pnlFilter.add(btnThucHienBaoCao);
        pnlFilter.add(btnDongBaoCao);

        add(pnlFilter, BorderLayout.NORTH);

        // 2. Vùng hiển thị Báo cáo chuẩn Crystal Report ở Trung tâm
        JPanel pnlCrystalReport = new JPanel(new BorderLayout(8, 8));
        pnlCrystalReport.setBackground(Color.WHITE);
        pnlCrystalReport.setBorder(new CompoundBorder(
                new EmptyBorder(4, 12, 12, 12),
                new CompoundBorder(new LineBorder(new Color(180, 180, 180), 1), new EmptyBorder(14, 22, 14, 22))
        ));

        // 2.1. Phần đầu báo cáo (Header + Thông tin Xe)
        JPanel pnlReportHeader = new JPanel(new BorderLayout(4, 8));
        pnlReportHeader.setBackground(Color.WHITE);

        JPanel pnlTitles = new JPanel(new GridLayout(2, 1, 0, 4));
        pnlTitles.setBackground(Color.WHITE);
        JLabel lblStore = new JLabel("Cửa hàng Mô tô PKL", SwingConstants.LEFT);
        lblStore.setFont(new Font("Times New Roman", Font.BOLD | Font.ITALIC, 15));
        JLabel lblMainReportTitle = new JLabel("THỐNG KÊ KHÁCH HÀNG ĐÃ MUA XE", SwingConstants.CENTER);
        lblMainReportTitle.setFont(new Font("Times New Roman", Font.BOLD, 24));
        pnlTitles.add(lblStore);
        pnlTitles.add(lblMainReportTitle);
        pnlReportHeader.add(pnlTitles, BorderLayout.NORTH);

        // Thông tin Xe được chọn: Mã xe, Tên xe, Đơn Giá, Dung tích
        JPanel pnlXeInfo = new JPanel(new GridBagLayout());
        pnlXeInfo.setBackground(Color.WHITE);
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(4, 8, 4, 8);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        Font lblBold = new Font("Times New Roman", Font.BOLD, 15);
        Font valFont = new Font("Times New Roman", Font.BOLD, 15);
        Color valBlue = new Color(25, 55, 150);

        lblValMaXe = taoNhanGiaTriBaoCao(valFont, valBlue);
        lblValTenXe = taoNhanGiaTriBaoCao(valFont, valBlue);
        lblValDonGia = taoNhanGiaTriBaoCao(valFont, valBlue);
        lblValDungTich = taoNhanGiaTriBaoCao(valFont, valBlue);

        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0;
```

```
        pnlXeInfo.add(taoNhanTieuDe("Mã xe:", lblBold), gbc);
        gbc.gridx = 1; gbc.weightx = 0.45;
        pnlXeInfo.add(lblValMaXe, gbc);

        gbc.gridx = 2; gbc.weightx = 0;
        pnlXeInfo.add(taoNhanTieuDe("Tên xe:", lblBold), gbc);
        gbc.gridx = 3; gbc.weightx = 0.55;
        pnlXeInfo.add(lblValTenXe, gbc);

        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0;
        pnlXeInfo.add(taoNhanTieuDe("Đơn Giá:", lblBold), gbc);
        gbc.gridx = 1; gbc.weightx = 0.45;
        pnlXeInfo.add(lblValDonGia, gbc);

        gbc.gridx = 2; gbc.weightx = 0;
        pnlXeInfo.add(taoNhanTieuDe("Dung tích:", lblBold), gbc);
        gbc.gridx = 3; gbc.weightx = 0.55;
        pnlXeInfo.add(lblValDungTich, gbc);

        pnlReportHeader.add(pnlXeInfo, BorderLayout.CENTER);
        pnlCrystalReport.add(pnlReportHeader, BorderLayout.NORTH);

        // 2.2. Bảng báo cáo 5 cột: STT | Mã khách hàng | Ngày mua | Số lượng | Ghi chú
        reportTableModel = new DefaultTableModel(
                new String[]{"STT", "Mã khách hàng", "Ngày mua", "Số lượng", "Ghi chú"}, 0
        ) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return false;
            }
        };
        tblBaoCao = new JTable(reportTableModel);
        tblBaoCao.setRowHeight(25);
        tblBaoCao.setFont(new Font("Times New Roman", Font.PLAIN, 15));
        tblBaoCao.setGridColor(new Color(180, 120, 140));
        tblBaoCao.setShowGrid(true);

        JTableHeader header = tblBaoCao.getTableHeader();
        header.setPreferredSize(new Dimension(100, 28));
        header.setDefaultRenderer(new DefaultTableCellRenderer() {
            @Override
            public Component getTableCellRendererComponent(JTable table, Object value,
                                                           boolean isSelected, boolean hasFocus, int row, int
column) {
                JLabel lbl = (JLabel) super.getTableCellRendererComponent(table, value, isSelected, hasFocus,
row, column);
                lbl.setBackground(new Color(82, 182, 72));
                lbl.setForeground(Color.WHITE);
                lbl.setFont(new Font("Times New Roman", Font.BOLD, 15));
                lbl.setHorizontalAlignment(SwingConstants.CENTER);
                lbl.setBorder(new LineBorder(new Color(160, 110, 130), 1));
                lbl.setOpaque(true);
                return lbl;
            }
        });

        DefaultTableCellRenderer centerRenderer = new DefaultTableCellRenderer();
        centerRenderer.setHorizontalAlignment(SwingConstants.CENTER);
        tblBaoCao.getColumnModel().getColumn(0).setCellRenderer(centerRenderer);
        tblBaoCao.getColumnModel().getColumn(1).setCellRenderer(centerRenderer);
        tblBaoCao.getColumnModel().getColumn(2).setCellRenderer(centerRenderer);
        tblBaoCao.getColumnModel().getColumn(3).setCellRenderer(centerRenderer);
```

```
tblBaoCao.getColumnModel().getColumn(0).setPreferredWidth(50);
tblBaoCao.getColumnModel().getColumn(1).setPreferredWidth(140);
tblBaoCao.getColumnModel().getColumn(2).setPreferredWidth(120);
tblBaoCao.getColumnModel().getColumn(3).setPreferredWidth(90);
tblBaoCao.getColumnModel().getColumn(4).setPreferredWidth(180);

JScrollPane scrollReport = new JScrollPane(tblBaoCao);
scrollReport.getViewport().setBackground(Color.WHITE);
pnlCrystalReport.add(scrollReport, BorderLayout.CENTER);

// 2.3. Chân trang Báo cáo (Số lượng khách hàng + Ngày tháng + Người lập)
JPanel pnlFooter = new JPanel(new BorderLayout(4, 6));
pnlFooter.setBackground(Color.WHITE);

JPanel pnlCount = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 2));
pnlCount.setBackground(Color.WHITE);
JLabel lblCountTitle = new JLabel("Số lượng khách hàng:");
lblCountTitle.setFont(lblBold);
lblValSoLuongKH = new JLabel("0");
lblValSoLuongKH.setFont(valFont);
lblValSoLuongKH.setForeground(valBlue);
lblValSoLuongKH.setBorder(new MatteBorder(0, 0, 1, 0, Color.DARK_GRAY));
pnlCount.add(lblCountTitle);
pnlCount.add(lblValSoLuongKH);
pnlFooter.add(pnlCount, BorderLayout.NORTH);

JPanel pnlSign = new JPanel(new GridLayout(4, 1, 0, 2));
pnlSign.setBackground(Color.WHITE);
pnlSign.setPreferredSize(new Dimension(280, 90));
String ngayHienTai = new SimpleDateFormat("'TP.HCM, ngày' dd 'tháng' MM 'năm' yyyy").format(new Date());
lblNgayLap = new JLabel(ngayHienTai, SwingConstants.CENTER);
lblNgayLap.setFont(new Font("Times New Roman", Font.BOLD, 14));
JLabel lblNguoiLap = new JLabel("Người lập", SwingConstants.CENTER);
lblNguoiLap.setFont(new Font("Times New Roman", Font.BOLD, 14));
JLabel lblBlank = new JLabel("", SwingConstants.CENTER);
JLabel lblTenNguoiLap = new JLabel("Trần Mạnh Hùng (AT21N0129)", SwingConstants.CENTER);
lblTenNguoiLap.setFont(new Font("Times New Roman", Font.BOLD, 14));
pnlSign.add(lblNgayLap);
pnlSign.add(lblNguoiLap);
pnlSign.add(lblBlank);
pnlSign.add(lblTenNguoiLap);

JPanel pnlSignWrap = new JPanel(new FlowLayout(FlowLayout.RIGHT));
pnlSignWrap.setBackground(Color.WHITE);
pnlSignWrap.add(pnlSign);
pnlFooter.add(pnlSignWrap, BorderLayout.CENTER);

pnlCrystalReport.add(pnlFooter, BorderLayout.SOUTH);
add(pnlCrystalReport, BorderLayout.CENTER);

// Sự kiện các nút trên cửa sổ Báo cáo (đều hỏi Yes/No theo Ghi chú 1)
btnThucHienBaoCao.addActionListener(e -> {
    if (xacNhan("Bạn có chắc chắn muốn thực hiện phát sinh báo cáo theo tiêu chí đã chọn hay không?")) {
        thucHienPhatSinhBaoCao(true);
    }
});

btnDongBaoCao.addActionListener(e -> {
    if (xacNhan("Bạn có chắc chắn muốn đóng Cửa sổ Báo cáo hay không?")) {
        dispose();
    }
```

```
        });
    }

    private static JLabel taoNhanTieuDe(String text, Font font) {
        JLabel lbl = new JLabel(text);
        lbl.setFont(font);
        return lbl;
    }

    private static JLabel taoNhanGiaTriBaoCao(Font font, Color color) {
        JLabel lbl = new JLabel(" ");
        lbl.setFont(font);
        lbl.setForeground(color);
        lbl.setBorder(new MatteBorder(0, 0, 1, 0, Color.GRAY));
        return lbl;
    }

    private boolean xacNhan(String message) {
        if (autoConfirmMode) {
            return true;
        }
        int ans = JOptionPane.showConfirmDialog(this, message, "Xác nhận thực thi",
                JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE);
        return ans == JOptionPane.YES_OPTION;
    }

    public void napDanhSachMaXe(String initialMaXe) {
        cboMaXe.removeAllItems();
        try {
            danhSachXe = repository.layDanhSachXe();
        } catch (SQLException e) {
            repository.khoiTaoDongDuLieuAo();
            try {
                danhSachXe = repository.layDanhSachXe();
            } catch (SQLException ignored) {
            }
        }
        for (Xe x : danhSachXe) {
            cboMaXe.addItem(x.getMaXe());
        }
        cboMaXe.addItem("-- Tất cả --");
        if (initialMaXe != null && !initialMaXe.trim().isEmpty()) {
            cboMaXe.setSelectedItem(initialMaXe.trim());
        } else if (cboMaXe.getItemCount() > 0) {
            cboMaXe.setSelectedIndex(0);
        }
    }

    public void thucHienPhatSinhBaoCao(boolean showCheckConnectionAlert) {
        if (!repository.kiemTraKetNoi()) {
            if (showCheckConnectionAlert && !autoConfirmMode) {
                JOptionPane.showMessageDialog(this,
                        "Kết nối Database Server không thành công!\nHệ thống sử dụng dòng dữ liệu ảo để hiển thị
báo cáo.",
                        "Cảnh báo kết nối CSDL", JOptionPane.WARNING_MESSAGE);
            }
            repository.khoiTaoDongDuLieuAo();
        }
        String selectedMaXe = cboMaXe.getSelectedItem() == null ? "" :
cboMaXe.getSelectedItem().toString().trim();
        String ngayMuaFilter = txtNgayMua.getText().trim();
```

```
        Xe xeInfo = null;
        for (Xe x : danhSachXe) {
            if (x.getMaXe().equalsIgnoreCase(selectedMaXe)) {
                xeInfo = x;
                break;
            }
        }
        if (xeInfo != null) {
            lblValMaXe.setText(xeInfo.getMaXe());
            lblValTenXe.setText(xeInfo.getTenXe());
            lblValDonGia.setText(Xe.dinhDangSo(xeInfo.getDonGia()) + " VNĐ");
            lblValDungTich.setText(String.valueOf(xeInfo.getDungTich()) + " cc");
        } else {
            lblValMaXe.setText("Tất cả mã xe");
            lblValTenXe.setText("Tất cả dòng xe");
            lblValDonGia.setText("---");
            lblValDungTich.setText("---");
        }

        reportTableModel.setRowCount(0);
        try {
            List<ChiTietMuaXe> rows = repository.locBaoCaoMuaXe(selectedMaXe, ngayMuaFilter);
            for (int i = 0; i < rows.size(); i++) {
                ChiTietMuaXe ct = rows.get(i);
                reportTableModel.addRow(new Object[]{
                        Integer.valueOf(i + 1),
                        ct.getMaKH(),
                        ct.getNgayMua(),
                        Integer.valueOf(ct.getSoLuong()),
                        ct.getGhiChu()
                });
            }
            lblValSoLuongKH.setText("  " + rows.size() + "  ");
        } catch (SQLException ex) {
            lblValSoLuongKH.setText("  0  ");
        }
    }

    // Các phương thức Getter chuẩn OOP phục vụ kiểm thử tự động
    public JComboBox<String> getCboMaXe() { return cboMaXe; }
    public JTextField getTxtNgayMua() { return txtNgayMua; }
    public JButton getBtnThucHienBaoCao() { return btnThucHienBaoCao; }
    public JTable getTblBaoCao() { return tblBaoCao; }
    public JLabel getLblValMaXe() { return lblValMaXe; }
    public JLabel getLblValTenXe() { return lblValTenXe; }
    public JLabel getLblValSoLuongKH() { return lblValSoLuongKH; }
}


/**
 * Cửa sổ chính chương trình: QUẢN LÝ KHÁCH HÀNG MUA XE (Kiểm tra cuối kỳ - Đề 01).
 *
 * Bám sát 100% kiến thức trong 7 tài liệu gốc của trường (AWT/Swing, OOP, Sự kiện,
 * JTable, JComboBox, SimpleDateFormat, JDBC SQL Server) và đáp ứng trọn vẹn:
 * - Lưu ý đầu đề: Kiểm tra kết nối CSDL (nếu không kết nối được thì đưa dữ liệu trống rỗng
 *   lên cửa sổ và thông báo kết nối CSDL không được).
 * - Câu 2 (a -> i): Giao diện, Khởi tạo & Ràng buộc bàn phím, Thống kê, Điều hướng Xe (1/4),
 *   Thêm Xe (báo lỗi "Mã xe đã tồn tại, vui lòng nhập lại!"), Xóa Xe (đưa Xe kế tiếp lên),
 *   Khung hiển thị dữ liệu chi tiết (cột STT đầu tiên + Thêm/Sửa/Xóa khách hàng mua xe),
 *   Cửa sổ Báo cáo (Combobox Mã xe + Txtbox Ngày mua + Btn Thực hiện báo cáo), Ghi file .txt.
 * - Ghi chú cuối đề: Mọi nút lệnh đều hỏi xác nhận Yes/No; tự động cập nhật Thống kê khi CSDL
 *   thay đổi; tự động tạo dòng dữ liệu ảo đưa lên các control khi xử lý mà mất kết nối DB.
```

```
 */
@SuppressWarnings("serial")
class QuanLyMuaBanXeGUI extends JFrame {
    private static final long serialVersionUID = 1L;

    public static boolean isTestMode = false;

    private final MuaBanXeRepository repository;
    private List<Xe> dsXe = new ArrayList<Xe>();
    private List<KhachHang> dsKhachHang = new ArrayList<KhachHang>();
    private List<ChiTietMuaXe> dsChiTietHienTai = new ArrayList<ChiTietMuaXe>();
    private int currentIndex = -1;
    private String lastAlertMessage = "";
    private BaoCaoMuaXeDialog lastReportDialog = null;

    // Các ô văn bản thông tin Xe trên Cửa sổ chính
    JTextField txtMaXe;
    JTextField txtTenXe;
    JTextField txtDungTich;
    JTextField txtDonGia;

    // Khung hiển thị dữ liệu chi tiết (JTable)
    DefaultTableModel detailTableModel;
    JTable tblChiTiet;
    JComboBox<String> cboTableMaKH;

    // Thanh nhập/cập nhật nhanh Khách hàng mua Xe (phục vụ Câu 2g: Thêm, Sửa, Xóa trên Khung chi tiết)
    JComboBox<String> cboMaKH;
    JTextField txtTenKH;
    JTextField txtCMND;
    JComboBox<String> cboGioiTinh;
    JTextField txtNgayMua;
    JTextField txtSoLuongMua;
    JTextField txtGhiChuMua;
    JButton btnThemKHMua;
    JButton btnSuaKHMua;
    JButton btnXoaKHMua;

    // 2 Textbox Thống kê của Cửa sổ (Câu 2c)
    JTextField txtTongSoKH;
    JTextField txtTongThanhTien;

    // Cụm điều hướng Xe (Câu 2d)
    JButton btnVeDau;
    JButton btnVeTruoc;
    JTextField txtChiSo;
    JButton btnVeSau;
    JButton btnVeCuoi;

    // Cụm 5 nút chức năng chính (Câu 2e, 2f, 2h, 2i)
    JButton btnThemXe;
    JButton btnXoaXe;
    JButton btnBaoCao;
    JButton btnGhiFile;
    JButton btnKetThuc;
    JButton btnNhapMoiXe;

    public QuanLyMuaBanXeGUI() {
        this(new MuaBanXeRepository());
    }

    public QuanLyMuaBanXeGUI(MuaBanXeRepository repository) {
```

```
        super("Trung tâm xe gắn máy NVT - Quản lý khách hàng mua xe");
        this.repository = repository == null ? new MuaBanXeRepository() : repository;
        khoiTaoGiaoDien();
        khoiTaoSuKien();
        kiemTraVaTaiDuLieuBanDau();
    }

    /**
     * Câu 2a: Thiết kế giao diện chuẩn theo hình chụp trong đề thi.
     */
    private void khoiTaoGiaoDien() {
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        setSize(920, 610);
        setMinimumSize(new Dimension(880, 580));
        setLocationRelativeTo(null);

        Color bgMain = new Color(242, 242, 242);
        JPanel root = new JPanel(new BorderLayout(0, 8));
        root.setBackground(bgMain);
        setContentPane(root);

        // 1. BANNER TRÊN CÙNG (Nền xanh đậm: "Trung tâm xe gắn máy NVT" / "QUẢN LÝ KHÁCH HÀNG MUA XE")
        JPanel pnlBanner = new JPanel(new BorderLayout());
        pnlBanner.setBackground(new Color(18, 53, 126));
        pnlBanner.setPreferredSize(new Dimension(860, 86));
        pnlBanner.setBorder(new EmptyBorder(6, 12, 8, 12));

        JLabel lblSubBanner = new JLabel("Trung tâm xe gắn máy NVT");
        lblSubBanner.setFont(new Font("Times New Roman", Font.BOLD, 16));
        lblSubBanner.setForeground(new Color(220, 232, 255));
        pnlBanner.add(lblSubBanner, BorderLayout.NORTH);

        JLabel lblMainBanner = new JLabel("QUẢN LÝ KHÁCH HÀNG MUA XE", SwingConstants.CENTER);
        lblMainBanner.setFont(new Font("Times New Roman", Font.BOLD, 30));
        lblMainBanner.setForeground(Color.WHITE);
        pnlBanner.add(lblMainBanner, BorderLayout.CENTER);

        root.add(pnlBanner, BorderLayout.NORTH);

        // 2. PHẦN THÂN CHÍNH (Thông tin Xe + Khung hiển thị dữ liệu chi tiết + Thống kê)
        JPanel pnlBody = new JPanel(new BorderLayout(6, 8));
        pnlBody.setBackground(bgMain);
        pnlBody.setBorder(new EmptyBorder(4, 18, 4, 18));

        // 2.1. Khung nhập thông tin Xe (Mã xe, Tên xe, Dung tích, Đơn giá)
        JPanel pnlXeForm = new JPanel(new GridBagLayout());
        pnlXeForm.setBackground(bgMain);
        pnlXeForm.setBorder(new EmptyBorder(2, 20, 4, 20));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(5, 8, 5, 8);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        Font fontLabel = new Font("Times New Roman", Font.BOLD, 16);
        Font fontInput = new Font("Tahoma", Font.BOLD, 14);
        Color textBlue = new Color(18, 53, 126);

        JLabel lblMaXe = new JLabel("Mã xe");
        lblMaXe.setFont(fontLabel);
        txtMaXe = new JTextField(14);
        txtMaXe.setFont(fontInput);
        txtMaXe.setForeground(textBlue);
```

```
        JLabel lblTenXe = new JLabel("Tên xe");
        lblTenXe.setFont(fontLabel);
        txtTenXe = new JTextField(20);
        txtTenXe.setFont(fontInput);
        txtTenXe.setForeground(textBlue);

        JLabel lblDungTich = new JLabel("Dung tích");
        lblDungTich.setFont(fontLabel);
        txtDungTich = taoOChiNhapSo("", 14, 6, false);
        txtDungTich.setFont(fontInput);
        txtDungTich.setForeground(textBlue);

        JLabel lblDonGia = new JLabel("Đơn giá");
        lblDonGia.setFont(fontLabel);
        txtDonGia = taoOChiNhapSo("", 20, 15, true);
        txtDonGia.setFont(fontInput);
        txtDonGia.setForeground(textBlue);

        btnNhapMoiXe = new JButton("Làm trống ô Xe");
        btnNhapMoiXe.setFont(new Font("Tahoma", Font.PLAIN, 11));
        btnNhapMoiXe.setToolTipText("Xóa trắng 4 ô Mã xe, Tên xe, Dung tích, Đơn giá để nhập Xe mới");

        // Dòng 1: Mã xe | Tên xe
        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0;
        pnlXeForm.add(lblMaXe, gbc);
        gbc.gridx = 1; gbc.weightx = 0.35;
        pnlXeForm.add(txtMaXe, gbc);
        gbc.gridx = 2; gbc.weightx = 0;
        gbc.insets = new Insets(5, 28, 5, 8);
        pnlXeForm.add(lblTenXe, gbc);
        gbc.gridx = 3; gbc.weightx = 0.55;
        gbc.insets = new Insets(5, 8, 5, 8);
        pnlXeForm.add(txtTenXe, gbc);

        // Dòng 2: Dung tích | Đơn giá
        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0;
        pnlXeForm.add(lblDungTich, gbc);
        gbc.gridx = 1; gbc.weightx = 0.35;
        pnlXeForm.add(txtDungTich, gbc);
        gbc.gridx = 2; gbc.weightx = 0;
        gbc.insets = new Insets(5, 28, 5, 8);
        pnlXeForm.add(lblDonGia, gbc);
        gbc.gridx = 3; gbc.weightx = 0.55;
        gbc.insets = new Insets(5, 8, 5, 8);
        pnlXeForm.add(txtDonGia, gbc);

        pnlBody.add(pnlXeForm, BorderLayout.NORTH);

        // 2.2. Khung hiển thị dữ liệu chi tiết (JTable 8 cột có STT đầu tiên + Thanh cập nhật Thêm/Sửa/Xóa KH
mua)
        JPanel pnlDetailContainer = new JPanel(new BorderLayout(0, 6));
        pnlDetailContainer.setBackground(bgMain);

        detailTableModel = new DefaultTableModel(
                new String[]{
                        "STT", "Mã KH", "Tên khách hàng", "Số CMND",
                        "Giới tính", "Ngày mua", "Số lượng", "Thành tiền"
                }, 0
        ) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return column == 1 || column == 2 || column == 3 || column == 4 || column == 5 || column == 6;
```

```
        }
    };

    tblChiTiet = new JTable(detailTableModel);
    tblChiTiet.setRowHeight(26);
    tblChiTiet.setFont(new Font("Times New Roman", Font.PLAIN, 15));
    tblChiTiet.setSelectionBackground(new Color(189, 215, 238));
    tblChiTiet.setSelectionForeground(new Color(12, 42, 110));
    tblChiTiet.setGridColor(new Color(180, 180, 180));
    tblChiTiet.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

    JTableHeader th = tblChiTiet.getTableHeader();
    th.setPreferredSize(new Dimension(100, 28));
    th.setFont(new Font("Times New Roman", Font.PLAIN, 15));
    th.setBackground(new Color(230, 230, 230));

    // Cột Mã KH có JComboBox chọn mã khách hàng giống hình đề
    cboTableMaKH = new JComboBox<String>();
    tblChiTiet.getColumnModel().getColumn(1).setCellEditor(new DefaultCellEditor(cboTableMaKH));

    DefaultTableCellRenderer centerCell = new DefaultTableCellRenderer();
    centerCell.setHorizontalAlignment(SwingConstants.CENTER);
    DefaultTableCellRenderer rightCell = new DefaultTableCellRenderer();
    rightCell.setHorizontalAlignment(SwingConstants.RIGHT);

    tblChiTiet.getColumnModel().getColumn(0).setCellRenderer(centerCell);
    tblChiTiet.getColumnModel().getColumn(1).setCellRenderer(centerCell);
    tblChiTiet.getColumnModel().getColumn(3).setCellRenderer(centerCell);
    tblChiTiet.getColumnModel().getColumn(4).setCellRenderer(centerCell);
    tblChiTiet.getColumnModel().getColumn(5).setCellRenderer(centerCell);
    tblChiTiet.getColumnModel().getColumn(6).setCellRenderer(centerCell);
    tblChiTiet.getColumnModel().getColumn(7).setCellRenderer(rightCell);

    tblChiTiet.getColumnModel().getColumn(0).setPreferredWidth(42);
    tblChiTiet.getColumnModel().getColumn(1).setPreferredWidth(78);
    tblChiTiet.getColumnModel().getColumn(2).setPreferredWidth(155);
    tblChiTiet.getColumnModel().getColumn(3).setPreferredWidth(105);
    tblChiTiet.getColumnModel().getColumn(4).setPreferredWidth(78);
    tblChiTiet.getColumnModel().getColumn(5).setPreferredWidth(98);
    tblChiTiet.getColumnModel().getColumn(6).setPreferredWidth(75);
    tblChiTiet.getColumnModel().getColumn(7).setPreferredWidth(115);

    JScrollPane scrollTable = new JScrollPane(tblChiTiet);
    scrollTable.setPreferredSize(new Dimension(790, 145));
    scrollTable.setBorder(new LineBorder(new Color(160, 160, 160), 1));
    pnlDetailContainer.add(scrollTable, BorderLayout.CENTER);

    // Thanh điều khiển cập nhật chi tiết Khách hàng mua Xe (Câu 2g: Thêm; Sửa; Xóa)
    JPanel pnlDetailCrud = new JPanel(new FlowLayout(FlowLayout.LEFT, 5, 4));
    pnlDetailCrud.setBackground(new Color(234, 238, 244));
    pnlDetailCrud.setBorder(new CompoundBorder(
            new LineBorder(new Color(195, 202, 212), 1),
            new EmptyBorder(2, 4, 2, 4)
    ));

    cboMaKH = new JComboBox<String>();
    cboMaKH.setEditable(true);
    cboMaKH.setPreferredSize(new Dimension(72, 24));
    txtTenKH = new JTextField("Nguyễn Văn Sơn", 11);
    txtCMND = taoOChiNhapSo("301458080", 8, 12, false);
    cboGioiTinh = new JComboBox<String>(new String[]{"Nam", "Nữ"});
    cboGioiTinh.setPreferredSize(new Dimension(60, 24));
```

```
        txtNgayMua = new JTextField("28/10/2011", 8);
        txtSoLuongMua = taoOChiNhapSo("1", 3, 4, false);
        txtGhiChuMua = new JTextField("", 7);

        btnThemKHMua = new JButton("+ Thêm KH");
        btnSuaKHMua = new JButton("Sửa KH");
        btnXoaKHMua = new JButton("- Xóa KH");
        Font smallBtnFont = new Font("Tahoma", Font.BOLD, 11);
        Insets smallMargin = new Insets(2, 6, 2, 6);
        for (JButton b : new JButton[]{btnThemKHMua, btnSuaKHMua, btnXoaKHMua}) {
            b.setFont(smallBtnFont);
            b.setMargin(smallMargin);
        }

        pnlDetailCrud.add(new JLabel("Mã KH:"));
        pnlDetailCrud.add(cboMaKH);
        pnlDetailCrud.add(new JLabel("Tên:"));
        pnlDetailCrud.add(txtTenKH);
        pnlDetailCrud.add(new JLabel("CMND:"));
        pnlDetailCrud.add(txtCMND);
        pnlDetailCrud.add(cboGioiTinh);
        pnlDetailCrud.add(new JLabel("Ngày:"));
        pnlDetailCrud.add(txtNgayMua);
        pnlDetailCrud.add(new JLabel("SL:"));
        pnlDetailCrud.add(txtSoLuongMua);
        pnlDetailCrud.add(btnThemKHMua);
        pnlDetailCrud.add(btnSuaKHMua);
        pnlDetailCrud.add(btnXoaKHMua);

        pnlDetailContainer.add(pnlDetailCrud, BorderLayout.SOUTH);
        pnlBody.add(pnlDetailContainer, BorderLayout.CENTER);

        // 2.3. Hàng Thống kê của Cửa sổ (Câu 2c: Tổng số khách hàng & Tổng thành tiền)
        JPanel pnlStats = new JPanel(new BorderLayout());
        pnlStats.setBackground(bgMain);
        pnlStats.setBorder(new EmptyBorder(6, 4, 2, 4));

        JPanel pnlLeftStat = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 0));
        pnlLeftStat.setBackground(bgMain);
        JLabel lblTongSoKH = new JLabel("Tổng số khách hàng:");
        lblTongSoKH.setFont(new Font("Times New Roman", Font.PLAIN, 16));
        lblTongSoKH.setForeground(new Color(90, 90, 90));
        txtTongSoKH = new JTextField("0", 5);
        txtTongSoKH.setEditable(false);
        txtTongSoKH.setBorder(BorderFactory.createEmptyBorder(2, 4, 2, 4));
        txtTongSoKH.setBackground(bgMain);
        txtTongSoKH.setFont(new Font("Times New Roman", Font.BOLD, 16));
        pnlLeftStat.add(lblTongSoKH);
        pnlLeftStat.add(txtTongSoKH);

        JPanel pnlRightStat = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
        pnlRightStat.setBackground(bgMain);
        JLabel lblTongThanhTien = new JLabel("Tổng thành tiền:");
        lblTongThanhTien.setFont(new Font("Times New Roman", Font.PLAIN, 16));
        lblTongThanhTien.setForeground(new Color(90, 90, 90));
        txtTongThanhTien = new JTextField("0 VNĐ", 13);
        txtTongThanhTien.setEditable(false);
        txtTongThanhTien.setHorizontalAlignment(SwingConstants.RIGHT);
        txtTongThanhTien.setBackground(new Color(38, 38, 38));
        txtTongThanhTien.setForeground(Color.WHITE);
        txtTongThanhTien.setFont(new Font("Times New Roman", Font.BOLD, 16));
        txtTongThanhTien.setBorder(new EmptyBorder(3, 8, 3, 8));
```

```
        pnlRightStat.add(lblTongThanhTien);
        pnlRightStat.add(txtTongThanhTien);

        pnlStats.add(pnlLeftStat, BorderLayout.WEST);
        pnlStats.add(pnlRightStat, BorderLayout.EAST);
        pnlBody.add(pnlStats, BorderLayout.SOUTH);

        root.add(pnlBody, BorderLayout.CENTER);

        // 3. THANH ĐIỀU HƯỚNG & 5 NÚT CHỨC NĂNG DƯỚI CÙNG (Câu 2d, 2e, 2f, 2h, 2i)
        JPanel pnlBottom = new JPanel(new BorderLayout(12, 0));
        pnlBottom.setBackground(bgMain);
        pnlBottom.setBorder(new EmptyBorder(4, 22, 16, 22));

        // Cụm điều hướng bên trái: |<, <, [1/4], >, >|
        JPanel pnlNav = new JPanel(new FlowLayout(FlowLayout.LEFT, 4, 0));
        pnlNav.setBackground(bgMain);
        Font navFont = new Font("Tahoma", Font.BOLD, 12);
        Dimension navDim = new Dimension(52, 36);
        Insets navMargin = new Insets(2, 4, 2, 4);

        btnVeDau = new JButton("|<");
        btnVeTruoc = new JButton("<");
        txtChiSo = new JTextField("0/0", 5);
        txtChiSo.setEditable(false);
        txtChiSo.setHorizontalAlignment(SwingConstants.CENTER);
        txtChiSo.setFont(new Font("Times New Roman", Font.PLAIN, 17));
        txtChiSo.setPreferredSize(new Dimension(68, 36));
        txtChiSo.setBackground(Color.WHITE);
        btnVeSau = new JButton(">");
        btnVeCuoi = new JButton(">|");

        for (JButton b : new JButton[]{btnVeDau, btnVeTruoc, btnVeSau, btnVeCuoi}) {
            b.setFont(navFont);
            b.setMargin(navMargin);
            b.setPreferredSize(navDim);
            b.setForeground(new Color(30, 70, 150));
        }

        pnlNav.add(btnVeDau);
        pnlNav.add(btnVeTruoc);
        pnlNav.add(txtChiSo);
        pnlNav.add(btnVeSau);
        pnlNav.add(btnVeCuoi);

        // Cụm 5 nút chức năng bên phải: Thêm Xe | Xóa xe | Báo cáo | Ghi file | Kết thúc
        JPanel pnlActions = new JPanel(new GridLayout(1, 5, 6, 0));
        pnlActions.setBackground(bgMain);
        Font actionFont = new Font("Times New Roman", Font.BOLD, 15);
        Dimension actionDim = new Dimension(92, 38);

        btnThemXe = new JButton("Thêm Xe");
        btnXoaXe = new JButton("Xóa xe");
        btnBaoCao = new JButton("Báo cáo");
        btnGhiFile = new JButton("Ghi file");
        btnKetThuc = new JButton("Kết thúc");

        for (JButton b : new JButton[]{btnThemXe, btnXoaXe, btnBaoCao, btnGhiFile, btnKetThuc}) {
            b.setFont(actionFont);
            b.setPreferredSize(actionDim);
            pnlActions.add(b);
        }
```

```
        pnlBottom.add(pnlNav, BorderLayout.WEST);
        pnlBottom.add(pnlActions, BorderLayout.EAST);

        root.add(pnlBottom, BorderLayout.SOUTH);
    }

    /**
     * Tạo JTextField chỉ nhận chữ số (và dấu phẩy nếu là ô Đơn giá), chặn cả phím gõ lẫn setText sai.
     */
    private static JTextField taoOChiNhapSo(String initial, int cols, final int maxLen, final boolean
allowCommas) {
        final JTextField field = new JTextField(initial, cols) {
            @Override
            public void setText(String t) {
                String val = t == null ? "" : t.trim();
                if (val.isEmpty()) {
                    super.setText("");
                    return;
                }
                String pattern = allowCommas ? "[0-9,]*" : "[0-9]*";
                if (!val.matches(pattern) || val.length() > maxLen) {
                    return;
                }
                super.setText(val);
            }
        };
        field.addKeyListener(new KeyAdapter() {
            @Override
            public void keyTyped(KeyEvent e) {
                char c = e.getKeyChar();
                if (Character.isISOControl(c)) {
                    return;
                }
                boolean validChar = Character.isDigit(c) || (allowCommas && c == ',');
                if (!validChar) {
                    e.consume();
                    return;
                }
                int sel = field.getSelectedText() == null ? 0 : field.getSelectedText().length();
                if (field.getText().length() - sel >= maxLen) {
                    e.consume();
                }
            }
        });
        return field;
    }

    /**
     * Câu 2b: Gắn sự kiện kiểm tra bàn phím, thoát control (FocusAdapter) và sự kiện các nút lệnh.
     */
    private void khoiTaoSuKien() {
        // Khi thoát khỏi các ô văn bản (focusLost): đánh dấu nền hồng nhạt nếu để trống
        JTextField[] requiredFields = new JTextField[]{txtMaXe, txtTenXe, txtDungTich, txtDonGia};
        for (final JTextField f : requiredFields) {
            f.addFocusListener(new FocusAdapter() {
                @Override
                public void focusLost(FocusEvent e) {
                    if (f.getText().trim().isEmpty()) {
                        f.setBackground(new Color(255, 235, 235));
                    } else {
                        f.setBackground(Color.WHITE);
```

```
            }
        }
    });
}

// Chọn dòng trên bảng Khung chi tiết -> đưa thông tin lên thanh cập nhật KH mua
tblChiTiet.getSelectionModel().addListSelectionListener(e -> {
    if (!e.getValueIsAdjusting()) {
        dongBoDongChonChiTietLenThanhKH();
    }
});

// Chọn Mã KH trên combobox -> tự động điền Tên KH, CMND, Giới tính
cboMaKH.addActionListener(e -> {
    Object sel = cboMaKH.getSelectedItem();
    if (sel != null) {
        String ma = sel.toString().trim();
        for (KhachHang kh : dsKhachHang) {
            if (kh.getMaKH().equalsIgnoreCase(ma)) {
                txtTenKH.setText(kh.getTenKH());
                txtCMND.setText(kh.getCmnd());
                cboGioiTinh.setSelectedItem(kh.getGioiTinh());
                break;
            }
        }
    }
});

// Điều hướng Xe (Câu 2d): Về đầu, Về trước, Về sau, Về cuối
btnVeDau.addActionListener(e -> {
    if (xacNhanThucThi("Bạn có chắc chắn muốn di chuyển về bản ghi Xe đầu tiên hay không?")) {
        diChuyenDenXe(0);
    }
});
btnVeTruoc.addActionListener(e -> {
    if (xacNhanThucThi("Bạn có chắc chắn muốn di chuyển về bản ghi Xe phía trước hay không?")) {
        diChuyenDenXe(currentIndex - 1);
    }
});
btnVeSau.addActionListener(e -> {
    if (xacNhanThucThi("Bạn có chắc chắn muốn di chuyển về bản ghi Xe kế tiếp hay không?")) {
        diChuyenDenXe(currentIndex + 1);
    }
});
btnVeCuoi.addActionListener(e -> {
    if (xacNhanThucThi("Bạn có chắc chắn muốn di chuyển về bản ghi Xe cuối cùng hay không?")) {
        diChuyenDenXe(dsXe.size() - 1);
    }
});

// Các nút chức năng chính (Câu 2e, 2f, 2h, 2i)
btnThemXe.addActionListener(e -> xuLyThemXe());
btnXoaXe.addActionListener(e -> xuLyXoaXe());
btnBaoCao.addActionListener(e -> xuLyMoBaoCao());
btnGhiFile.addActionListener(e -> xuLyGhiFile());
btnKetThuc.addActionListener(e -> xuLyKetThuc());

// Các nút Thêm / Sửa / Xóa Khách hàng mua Xe trên Khung hiển thị chi tiết (Câu 2g)
btnThemKHMua.addActionListener(e -> xuLyThemKhachHangMuaXe());
btnSuaKHMua.addActionListener(e -> xuLySuaKhachHangMuaXe());
btnXoaKHMua.addActionListener(e -> xuLyXoaKhachHangMuaXe());
```

```
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                xuLyKetThuc();
            }
        });
    }

    /**
     * Lưu ý đầu đề & Câu 2b:
     * Kiểm tra kết nối CSDL đã có kết nối chưa:
     * - Nếu kết nối không được thì đưa dữ liệu trống rỗng lên màn hình cửa sổ và thông báo là kết nối CSDL
không được.
     * - Nếu kết nối thành công thì lấy dữ liệu từ Database Server đưa lên control của Cửa sổ và Khung chi tiết.
     */
    public void kiemTraVaTaiDuLieuBanDau() {
        if (!repository.kiemTraKetNoi()) {
            duaDuLieuTrongRongLenCuaSo();
            thongBao("Kết nối CSDL không được! Đã đưa dữ liệu trống rỗng lên màn hình cửa sổ.");
            SwingUtilities.invokeLater(() -> txtMaXe.requestFocusInWindow());
            return;
        }
        taiDuLieuTuCSDLVaHienThi(0);
        // Câu 2b: Khởi tạo cửa sổ chính, con trỏ văn bản đặt vào ô Mã xe
        SwingUtilities.invokeLater(() -> txtMaXe.requestFocusInWindow());
    }

    /**
     * Đưa dữ liệu trống rỗng lên màn hình cửa sổ khi kiểm tra kết nối CSDL lúc khởi tạo thất bại.
     */
    public void duaDuLieuTrongRongLenCuaSo() {
        dsXe.clear();
        dsKhachHang.clear();
        dsChiTietHienTai.clear();
        currentIndex = -1;

        txtMaXe.setText("");
        txtTenXe.setText("");
        txtDungTich.setText("");
        txtDonGia.setText("");
        detailTableModel.setRowCount(0);
        txtTongSoKH.setText("0");
        txtTongThanhTien.setText("0 VNĐ");
        txtChiSo.setText("0/0");
        capNhatTrangThaiNutDieuHuong();
    }

    /**
     * Ghi chú 3 (Trang 3):
     * Khi xử lý dữ liệu trên Database Server thì luôn kiểm tra việc kết nối thành công hay không;
     * nếu kết nối dữ liệu không thành công thì tạo ra một dòng dữ liệu ảo, rồi sau đó đưa lên
     * các control chứa dữ liệu thích hợp trên Cửa sổ.
     */
    public boolean kiemTraKetNoiHoacTaoDongDuLieuAo() {
        if (repository.kiemTraKetNoi()) {
            return true;
        }
        repository.khoiTaoDongDuLieuAo();
        try {
            dsXe = repository.layDanhSachXe();
            dsKhachHang = repository.layDanhSachKhachHang();
            napDanhSachMaKHVaoCombo();
```

```
            if (!dsXe.isEmpty()) {
                hienThiXeTaiViTri(0);
            }
        } catch (SQLException ignored) {
        }
        thongBao("Kết nối Database Server không thành công! Đã tạo một dòng dữ liệu ảo và đưa lên các control
trên Cửa sổ.");
        return false;
    }

    public void taiDuLieuTuCSDLVaHienThi(int targetIndex) {
        try {
            dsXe = repository.layDanhSachXe();
            dsKhachHang = repository.layDanhSachKhachHang();
            napDanhSachMaKHVaoCombo();
            if (dsXe.isEmpty()) {
                duaDuLieuTrongRongLenCuaSo();
            } else {
                int idx = Math.max(0, Math.min(targetIndex, dsXe.size() - 1));
                hienThiXeTaiViTri(idx);
            }
        } catch (SQLException ex) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
        }
    }

    private void napDanhSachMaKHVaoCombo() {
        cboMaKH.removeAllItems();
        cboTableMaKH.removeAllItems();
        for (KhachHang kh : dsKhachHang) {
            cboMaKH.addItem(kh.getMaKH());
            cboTableMaKH.addItem(kh.getMaKH());
        }
    }

    /**
     * Hiển thị bản ghi Xe tại vị trí index lên các control của Cửa sổ và nạp Khung hiển thị dữ liệu chi tiết.
     */
    public void hienThiXeTaiViTri(int index) {
        if (dsXe.isEmpty() || index < 0 || index >= dsXe.size()) {
            duaDuLieuTrongRongLenCuaSo();
            return;
        }
        currentIndex = index;
        Xe xe = dsXe.get(currentIndex);
        txtMaXe.setText(xe.getMaXe());
        txtMaXe.setBackground(Color.WHITE);
        txtTenXe.setText(xe.getTenXe());
        txtTenXe.setBackground(Color.WHITE);
        txtDungTich.setText(String.valueOf(xe.getDungTich()));
        txtDungTich.setBackground(Color.WHITE);
        txtDonGia.setText(Xe.dinhDangSo(xe.getDonGia()));
        txtDonGia.setBackground(Color.WHITE);

        txtChiSo.setText((currentIndex + 1) + "/" + dsXe.size());
        capNhatTrangThaiNutDieuHuong();
        napBangChiTietVaThongKe(xe.getMaXe());
    }

    /**
     * Câu 2c & 2g: Nạp bảng Khung hiển thị dữ liệu chi tiết (có cột STT đầu tiên)
     * và tính lại 2 control Thống kê (Tổng số khách hàng = n, Tổng thành tiền).
```

```
     */
    public void napBangChiTietVaThongKe(String maXe) {
        detailTableModel.setRowCount(0);
        dsChiTietHienTai.clear();
        try {
            dsChiTietHienTai = repository.layChiTietMuaTheoXe(maXe);
        } catch (SQLException ex) {
            dsChiTietHienTai = new ArrayList<ChiTietMuaXe>();
        }

        long tongThanhTien = 0L;
        for (int i = 0; i < dsChiTietHienTai.size(); i++) {
            ChiTietMuaXe ct = dsChiTietHienTai.get(i);
            long thanhTien = ct.getThanhTien();
            tongThanhTien += thanhTien;
            detailTableModel.addRow(new Object[]{
                    Integer.valueOf(i + 1),
                    ct.getMaKH(),
                    ct.getTenKH(),
                    ct.getCmnd(),
                    ct.getGioiTinh(),
                    ct.getNgayMua(),
                    Integer.valueOf(ct.getSoLuong()),
                    Xe.dinhDangSo(thanhTien)
            });
        }

        txtTongSoKH.setText(String.valueOf(dsChiTietHienTai.size()));
        txtTongThanhTien.setText(Xe.dinhDangSo(tongThanhTien) + " VNĐ");

        if (!dsChiTietHienTai.isEmpty()) {
            tblChiTiet.setRowSelectionInterval(0, 0);
            dongBoDongChonChiTietLenThanhKH();
        }
    }

    private void dongBoDongChonChiTietLenThanhKH() {
        int row = tblChiTiet.getSelectedRow();
        if (row >= 0 && row < dsChiTietHienTai.size()) {
            ChiTietMuaXe ct = dsChiTietHienTai.get(row);
            cboMaKH.setSelectedItem(ct.getMaKH());
            txtTenKH.setText(ct.getTenKH());
            txtCMND.setText(ct.getCmnd());
            cboGioiTinh.setSelectedItem(ct.getGioiTinh());
            txtNgayMua.setText(ct.getNgayMua());
            txtSoLuongMua.setText(String.valueOf(ct.getSoLuong()));
            txtGhiChuMua.setText(ct.getGhiChu());
        }
    }

    private void capNhatTrangThaiNutDieuHuong() {
        int n = dsXe.size();
        btnVeDau.setEnabled(n > 0 && currentIndex > 0);
        btnVeTruoc.setEnabled(n > 0 && currentIndex > 0);
        btnVeSau.setEnabled(n > 0 && currentIndex >= 0 && currentIndex < n - 1);
        btnVeCuoi.setEnabled(n > 0 && currentIndex >= 0 && currentIndex < n - 1);
    }

    public void diChuyenDenXe(int newIndex) {
        if (newIndex >= 0 && newIndex < dsXe.size()) {
            hienThiXeTaiViTri(newIndex);
        }
```

```
    }

    /**
     * Kiểm tra ràng buộc các ô nhập Xe trên Cửa sổ chính (Câu 2b & 2e).
     */
    public Xe kiemTraVaDocThongTinXeTuForm() {
        String maXe = txtMaXe.getText().trim();
        if (maXe.isEmpty()) {
            txtMaXe.requestFocusInWindow();
            throw new IllegalArgumentException("Mã xe không được phép để trống (rỗng)!");
        }
        String tenXe = txtTenXe.getText().trim();
        if (tenXe.isEmpty()) {
            txtTenXe.requestFocusInWindow();
            throw new IllegalArgumentException("Tên xe không được phép để trống (rỗng)!");
        }
        String dungTichStr = txtDungTich.getText().trim();
        if (dungTichStr.isEmpty()) {
            txtDungTich.requestFocusInWindow();
            throw new IllegalArgumentException("Dung tích không được phép để trống (rỗng)!");
        }
        int dungTich;
        try {
            dungTich = Integer.parseInt(dungTichStr);
            if (dungTich <= 0) {
                throw new NumberFormatException();
            }
        } catch (NumberFormatException ex) {
            txtDungTich.requestFocusInWindow();
            throw new IllegalArgumentException("Dung tích phải là số nguyên dương!");
        }

        String donGiaStr = txtDonGia.getText().trim();
        if (donGiaStr.isEmpty()) {
            txtDonGia.requestFocusInWindow();
            throw new IllegalArgumentException("Đơn giá không được phép để trống (rỗng)!");
        }
        long donGia;
        try {
            donGia = Xe.parseDonGia(donGiaStr);
            if (donGia <= 0) {
                throw new NumberFormatException();
            }
        } catch (NumberFormatException ex) {
            txtDonGia.requestFocusInWindow();
            throw new IllegalArgumentException("Đơn giá phải là số nguyên dương!");
        }
        return new Xe(maXe, tenXe, dungTich, donGia);
    }

    /**
     * Câu 2e: Btn Thêm xe:
     * - Hỏi xác nhận Yes/No.
     * - Kiểm tra thông tin ràng buộc không rỗng.
     * - Kiểm tra Mã xe đã tồn tại trong table Xe chưa; nếu tồn tại thì thông báo lỗi:
     *   "Mã xe đã tồn tại, vui lòng nhập lại!"
     * - Ngược lại lưu thông tin Xe (và dòng khách hàng chọn mua hiện thời nếu hợp lệ) vào Database Server,
     *   sau đó cập nhật các control Thống kê và Khung hiển thị dữ liệu chi tiết.
     */
    public void xuLyThemXe() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn thực thi chức năng Thêm Xe hay không?")) {
            return;
```

```
        }
        if (!repository.kiemTraKetNoi()) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
            return;
        }
        Xe xeMoi;
        try {
            xeMoi = kiemTraVaDocThongTinXeTuForm();
        } catch (IllegalArgumentException ex) {
            thongBao(ex.getMessage());
            return;
        }

        try {
            if (repository.tonTaiMaXe(xeMoi.getMaXe())) {
                txtMaXe.requestFocusInWindow();
                txtMaXe.selectAll();
                thongBao("Mã xe đã tồn tại, vui lòng nhập lại!");
                return;
            }
            repository.themXe(xeMoi);

            // Nếu thanh thông tin Khách hàng mua xe đang có dữ liệu hợp lệ thì lưu kèm giao dịch mua xe
            String maKH = cboMaKH.getSelectedItem() == null ? "" : cboMaKH.getSelectedItem().toString().trim();
            String tenKH = txtTenKH.getText().trim();
            String cmnd = txtCMND.getText().trim();
            String ngayMua = txtNgayMua.getText().trim();
            String soLuongStr = txtSoLuongMua.getText().trim();
            if (!maKH.isEmpty() && !tenKH.isEmpty() && !cmnd.isEmpty() && !ngayMua.isEmpty() &&
!soLuongStr.isEmpty()) {
                try {
                    int sl = Integer.parseInt(soLuongStr);
                    if (sl > 0) {
                        ChiTietMuaXe ct = new ChiTietMuaXe(
                                maKH, tenKH, cmnd,
                                cboGioiTinh.getSelectedItem() == null ? "Nam" :
cboGioiTinh.getSelectedItem().toString(),
                                "TP.HCM", xeMoi.getMaXe(), ngayMua, sl, xeMoi.getDonGia(),
txtGhiChuMua.getText().trim()
                        );
                        repository.themChiTietMuaXe(ct);
                    }
                } catch (Exception ignored) {
                }
            }

            dsXe = repository.layDanhSachXe();
            int newIdx = dsXe.size() - 1;
            for (int i = 0; i < dsXe.size(); i++) {
                if (dsXe.get(i).getMaXe().equalsIgnoreCase(xeMoi.getMaXe())) {
                    newIdx = i;
                    break;
                }
            }
            hienThiXeTaiViTri(newIdx);
            thongBao("Thêm Xe '" + xeMoi.getMaXe() + "' vào Database Server thành công!");
        } catch (SQLException ex) {
            thongBao(ex.getMessage());
        }
    }

    /**
```

```
     * Câu 2f: Btn Xóa Xe:
     * - Hỏi xác nhận Yes/No.
     * - Xóa dòng dữ liệu hiện thời trên Cửa sổ (thông tin Xe) và cập nhật vào Database Server.
     * - Sau đó đưa dữ liệu của Xe kế tiếp lên Cửa sổ (cập nhật lại các control Thống kê).
     */
    public void xuLyXoaXe() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn xóa Xe hiện thời khỏi Database Server hay không?")) {
            return;
        }
        if (!repository.kiemTraKetNoi()) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
            return;
        }
        String maXeHienTai = txtMaXe.getText().trim();
        if (maXeHienTai.isEmpty() || dsXe.isEmpty()) {
            thongBao("Không có bản ghi Xe hiện thời để xóa!");
            return;
        }
        int viTriKeTiep = currentIndex;
        try {
            repository.xoaXe(maXeHienTai);
            dsXe = repository.layDanhSachXe();
            if (dsXe.isEmpty()) {
                duaDuLieuTrongRongLenCuaSo();
            } else {
                if (viTriKeTiep >= dsXe.size()) {
                    viTriKeTiep = dsXe.size() - 1;
                }
                hienThiXeTaiViTri(viTriKeTiep);
            }
            thongBao("Đã xóa Xe '" + maXeHienTai + "' và đưa dữ liệu Xe kế tiếp lên Cửa sổ.");
        } catch (SQLException ex) {
            thongBao("Lỗi khi xóa Xe: " + ex.getMessage());
        }
    }

    /**
     * Câu 2g: Thêm / Sửa / Xóa thông tin Khách hàng đã mua Xe đã chọn trên Khung hiển thị dữ liệu chi tiết.
     */
    public void xuLyThemKhachHangMuaXe() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn thêm Khách hàng mua Xe này vào CSDL hay không?")) {
            return;
        }
        if (!repository.kiemTraKetNoi()) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
            return;
        }
        if (currentIndex < 0 || currentIndex >= dsXe.size()) {
            thongBao("Vui lòng chọn một Xe trước khi thêm Khách hàng mua!");
            return;
        }
        try {
            ChiTietMuaXe ct = docChiTietMuaTuThanhKH();
            repository.themChiTietMuaXe(ct);
            dsKhachHang = repository.layDanhSachKhachHang();
            napDanhSachMaKHVaoCombo();
            napBangChiTietVaThongKe(dsXe.get(currentIndex).getMaXe());
            thongBao("Đã thêm Khách hàng '" + ct.getMaKH() + "' mua xe '" + ct.getMaXe() + "' thành công!");
        } catch (Exception ex) {
            thongBao(ex.getMessage());
        }
    }
```

```
    public void xuLySuaKhachHangMuaXe() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn cập nhật (Sửa) dòng Khách hàng mua Xe đang chọn hay không?"))
{
            return;
        }
        if (!repository.kiemTraKetNoi()) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
            return;
        }
        int row = tblChiTiet.getSelectedRow();
        if (row < 0 || row >= dsChiTietHienTai.size()) {
            thongBao("Vui lòng chọn một dòng Khách hàng trên Khung hiển thị chi tiết để sửa!");
            return;
        }
        ChiTietMuaXe banGoc = dsChiTietHienTai.get(row);
        try {
            ChiTietMuaXe capNhat = docChiTietMuaTuThanhKH();
            repository.suaChiTietMuaXe(banGoc.getMaKH(), banGoc.getMaXe(), banGoc.getNgayMua(), capNhat);
            dsKhachHang = repository.layDanhSachKhachHang();
            napDanhSachMaKHVaoCombo();
            napBangChiTietVaThongKe(banGoc.getMaXe());
            thongBao("Đã cập nhật thông tin Khách hàng mua xe thành công!");
        } catch (Exception ex) {
            thongBao(ex.getMessage());
        }
    }

    public void xuLyXoaKhachHangMuaXe() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn xóa dòng Khách hàng mua Xe đang chọn hay không?")) {
            return;
        }
        if (!repository.kiemTraKetNoi()) {
            kiemTraKetNoiHoacTaoDongDuLieuAo();
            return;
        }
        int row = tblChiTiet.getSelectedRow();
        if (row < 0 || row >= dsChiTietHienTai.size()) {
            thongBao("Vui lòng chọn một dòng Khách hàng trên Khung hiển thị chi tiết để xóa!");
            return;
        }
        ChiTietMuaXe ct = dsChiTietHienTai.get(row);
        try {
            repository.xoaChiTietMuaXe(ct.getMaKH(), ct.getMaXe(), ct.getNgayMua());
            napBangChiTietVaThongKe(ct.getMaXe());
            thongBao("Đã xóa Khách hàng '" + ct.getMaKH() + "' khỏi danh sách mua xe '" + ct.getMaXe() + "'.");
        } catch (Exception ex) {
            thongBao(ex.getMessage());
        }
    }

    private ChiTietMuaXe docChiTietMuaTuThanhKH() throws SQLException {
        Xe xe = dsXe.get(currentIndex);
        String maKH = cboMaKH.getSelectedItem() == null ? "" : cboMaKH.getSelectedItem().toString().trim();
        String tenKH = txtTenKH.getText().trim();
        String cmnd = txtCMND.getText().trim();
        String gioiTinh = cboGioiTinh.getSelectedItem() == null ? "Nam" :
cboGioiTinh.getSelectedItem().toString().trim();
        String ngayMua = txtNgayMua.getText().trim();
        String slStr = txtSoLuongMua.getText().trim();

        if (maKH.isEmpty() || tenKH.isEmpty() || cmnd.isEmpty() || ngayMua.isEmpty() || slStr.isEmpty()) {
```

```
            throw new IllegalArgumentException("Thông tin Khách hàng mua xe (Mã KH, Tên KH, CMND, Ngày mua, Số
lượng) không được để trống!");
        }
        MuaBanXeRepository.chuyenSangIsoDate(ngayMua); // Kiểm tra định dạng dd/MM/yyyy
        int soLuong;
        try {
            soLuong = Integer.parseInt(slStr);
            if (soLuong <= 0) {
                throw new NumberFormatException();
            }
        } catch (NumberFormatException ex) {
            throw new IllegalArgumentException("Số lượng mua phải là số nguyên dương!");
        }
        return new ChiTietMuaXe(maKH, tenKH, cmnd, gioiTinh, "TP.HCM",
                xe.getMaXe(), ngayMua, soLuong, xe.getDonGia(), txtGhiChuMua.getText().trim());
    }

    /**
     * Câu 2h: Btn Báo cáo:
     * Xuất hiện Cửa sổ Báo cáo có Combobox "Mã xe", Txtbox "Ngày mua", và Btn "Thực hiện báo cáo".
     */
    public void xuLyMoBaoCao() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn mở Cửa sổ Báo cáo hay không?")) {
            return;
        }
        String maXeHienTai = txtMaXe.getText().trim();
        lastReportDialog = new BaoCaoMuaXeDialog(this, repository, maXeHienTai, isTestMode);
        lastReportDialog.setVisible(true);
    }

    /**
     * Câu 2i: Btn Ghi file:
     * Tạo một file .txt trên thiết bị lưu trữ dữ liệu để lưu thông tin trên Khung hiển thị dữ liệu chi tiết
hiện thời.
     */
    public void xuLyGhiFile() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn ghi thông tin Khung hiển thị dữ liệu chi tiết hiện thời ra
file txt hay không?")) {
            return;
        }
        File targetFile;
        if (isTestMode) {
            targetFile = new File("ThongKeKhachHangMuaXe_" + txtMaXe.getText().trim() + ".txt");
        } else {
            JFileChooser chooser = new JFileChooser();
            chooser.setDialogTitle("Lưu Khung hiển thị dữ liệu chi tiết ra file TXT");
            chooser.setSelectedFile(new File("KhachHangMuaXe_" + txtMaXe.getText().trim() + ".txt"));
            if (chooser.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) {
                return;
            }
            targetFile = chooser.getSelectedFile();
        }
        luuChiTietRaFileTxt(targetFile);
    }

    public void luuChiTietRaFileTxt(File file) {
        try (PrintWriter out = new PrintWriter(file, StandardCharsets.UTF_8.name())) {

out.println("=============================================================================================");
            out.println("                  TRUNG TÂM XE GẮN MÁY NVT - KHUNG HIỂN THỊ DỮ LIỆU CHI TIẾT
");
```

```
out.println("=============================================================================================");
            out.println("Mã xe      : " + txtMaXe.getText().trim());
            out.println("Tên xe     : " + txtTenXe.getText().trim());
            out.println("Dung tích  : " + txtDungTich.getText().trim());
            out.println("Đơn giá    : " + txtDonGia.getText().trim() + " VNĐ");
            out.println("---------------------------------------------------------------------------------------
-------");
            out.printf("%-5s | %-8s | %-22s | %-12s | %-9s | %-12s | %-8s | %-15s%n",
                    "STT", "Mã KH", "Tên khách hàng", "Số CMND", "Giới tính", "Ngày mua", "Số lượng", "Thành
tiền");
            out.println("---------------------------------------------------------------------------------------
-------");
            for (int i = 0; i < dsChiTietHienTai.size(); i++) {
                ChiTietMuaXe ct = dsChiTietHienTai.get(i);
                out.printf("%-5d | %-8s | %-22s | %-12s | %-9s | %-12s | %-8d | %-15s%n",
                        i + 1,
                        ct.getMaKH(),
                        ct.getTenKH(),
                        ct.getCmnd(),
                        ct.getGioiTinh(),
                        ct.getNgayMua(),
                        ct.getSoLuong(),
                        Xe.dinhDangSo(ct.getThanhTien()));
            }

out.println("=============================================================================================");
            out.println("Tổng số khách hàng : " + txtTongSoKH.getText().trim());
            out.println("Tổng thành tiền    : " + txtTongThanhTien.getText().trim());

out.println("=============================================================================================");
            thongBao("Đã ghi dữ liệu Khung chi tiết ra file thành công:\n" + file.getAbsolutePath());
        } catch (Exception ex) {
            thongBao("Lỗi khi ghi file txt: " + ex.getMessage());
        }
    }

    /**
     * Nút Kết thúc: Hỏi xác nhận Yes/No trước khi đóng chương trình.
     */
    public void xuLyKetThuc() {
        if (!xacNhanThucThi("Bạn có chắc chắn muốn kết thúc và thoát chương trình hay không?")) {
            return;
        }
        if (lastReportDialog != null && lastReportDialog.isDisplayable()) {
            lastReportDialog.dispose();
        }
        dispose();
        if (!isTestMode) {
            System.exit(0);
        }
    }

    /**
     * Ghi chú 1 (Trang 3):
     * Tất cả các btn trên các Cửa sổ thì đều có xuất hiện thông báo hỏi người dùng có chắc chắn
     * thực thi tiếp hay không? Yes: đồng ý, No: không.
     */
    private boolean xacNhanThucThi(String message) {
        if (isTestMode) {
            return true;
        }
        int choice = JOptionPane.showConfirmDialog(
```

```
                this,
                message,
                "Xác nhận thực thi",
                JOptionPane.YES_NO_OPTION,
                JOptionPane.QUESTION_MESSAGE
        );
        return choice == JOptionPane.YES_OPTION;
    }

    private void thongBao(String message) {
        this.lastAlertMessage = message;
        if (!isTestMode) {
            JOptionPane.showMessageDialog(this, message, "Thông báo", JOptionPane.INFORMATION_MESSAGE);
        }
    }

    // Các phương thức Getter chuẩn OOP phục vụ kiểm thử tự động (không dùng Reflection)
    public String getLastAlertMessage() { return lastAlertMessage; }
    public BaoCaoMuaXeDialog getLastReportDialog() { return lastReportDialog; }
    public List<Xe> getDsXe() { return dsXe; }
    public List<ChiTietMuaXe> getDsChiTietHienTai() { return dsChiTietHienTai; }
    public int getCurrentIndex() { return currentIndex; }

    public static void khoiChayGiaoDien(String[] args) {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception ignored) {
        }
        SwingUtilities.invokeLater(() -> new QuanLyMuaBanXeGUI().setVisible(true));
    }
}
```